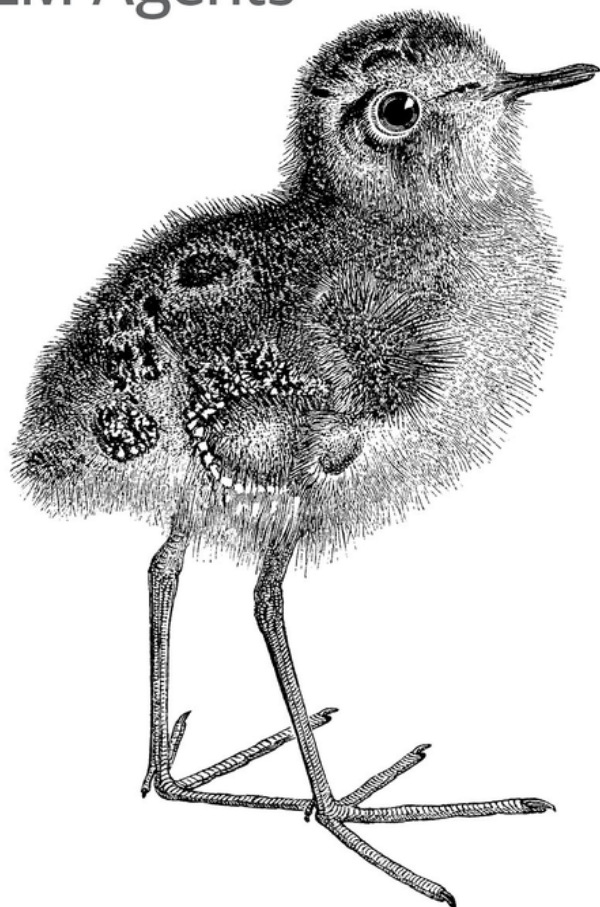


O'REILLY®

RAG with Python Cookbook

Practical Recipes from Data Preprocessing
to LLM Agents



**Early
Release**

**RAW &
UNEDITED**

Dominik Polzer

RAG with Python Cookbook

Practical Recipes from Data Preprocessing to LLM
Agents

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Dominik Polzer

O'REILLY®

Beijing • Boston • Farnham • Sebastopol • Tokyo

OceanofPDF.com

RAG with Python Cookbook

by Dominik Polzer

Copyright © 2026 Dominik Polzer. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Editors: Jill Leonard and Nicole Butterfield

Production Editor: Aleeya Rahman

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

April 2026: First Edition

Revision History for the Early Release

- 2025-03-11: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341600560> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *RAG with Python Cookbook*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-60051-5

OceanofPDF.com

Brief Table of Contents *(Not Yet Final)*

Preface (unavailable)

Chapter 1: Loading Data (available)

Chapter 2: Data Preparation (available)

Chapter 3: Embeddings (unavailable)

Chapter 4: Similarity Search (unavailable)

Chapter 5: Retrieval (unavailable)

Chapter 6: Prompt Engineering (unavailable)

Chapter 7: Generation (unavailable)

Chapter 8: Evaluating RAG Systems (unavailable)

Chapter 9: Agentic RAG (unavailable)

Chapter 10: GraphRAG (unavailable)

Chapter 11: RAG Apps (unavailable)

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 1. Loading Data

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at dmnkplzr@gmail.com.

In traditional companies, around **80% of information is unstructured**, buried within PowerPoint presentations, Word documents, Excel files, emails, and meeting records. **Figure 1-1** provides an overview of common file types and data sources for structured and unstructured data.

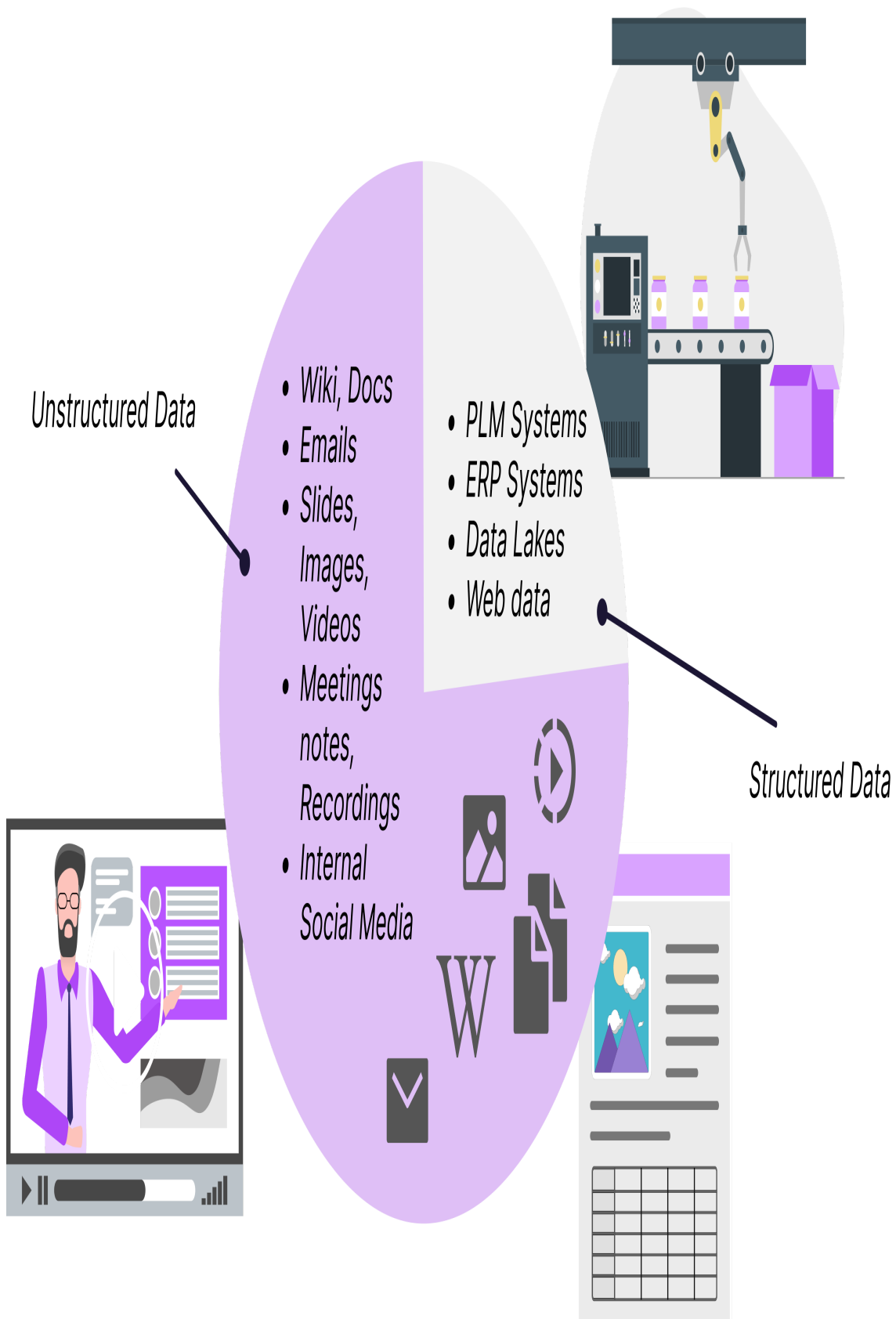


Figure 1-1. A common distribution of data in companies.

Foundation models can help preserve this information. Nowadays, we have a whole range of different foundation models available that mimic the way humans work with data. For example, large Language Models (LLMs) excel at tasks like generating text summaries, translations, and even code. Multimodal models take it further by handling various input formats, including text, audio, images, and videos. But the real unsung heroes are embedding models. These models convert the meaning of words and sentences into multidimensional vectors, crucial for many Machine Learning tasks like classification, clustering, and recommendation systems.

The capabilities of those models are remarkable, but they have one major limitation: each model has a maximum context size. The context size defines how much content the model can process in a single interaction. This limit is getting bigger and bigger, but building huge prompts leads to long response times for the models, making our application slow and costly.

That's a common hurdle for many LLM apps. Even if they provide more natural, human-like responses, they're still competing with traditional search engines when it comes to user expectations and attention spans. Nobody will wait for 10 seconds for a well-crafted answer when they can get the same info almost instantly from a regular search engine.

This is where RAG comes into play. RAG combines the strengths of search engines with the capabilities of foundation models. As shown in [Figure 1-2](#), the basic RAG system is divided into two parts.

First, we have the processing and indexing steps, which load all our available text content into our vector store. Here's how it works step by step:

1. **Load data:** Load data from various sources (Word, PDF, Excel, SQL, etc.).
2. **Split text:** Split the text into smaller chunks. Since embedding models and LLMs have a maximum context size, we divide the text to stay within this limit.

3. **Translate to embeddings:** Translate these text chunks into embeddings. These embeddings are vector representations of the text chunks, allowing us to calculate similarities between them.

Once all the data is loaded into the vector store, the app can use it during runtime. When a user asks a question, the system follows these steps:

4. **Translate question:** Translate the question into an embedding using the same model as during the loading process.
5. **Search for similarities:** Search for the most similar text chunks in the vector store by calculating the distance between the question embedding and the text chunk embeddings.
6. **Answer question:** The LLM then answers the question based on the relevant text chunks found in our database.

In essence, a RAG system combines a search engine and a LLM. The search engine finds the most relevant information in our database, and the LLM generates the answer to the user's question.

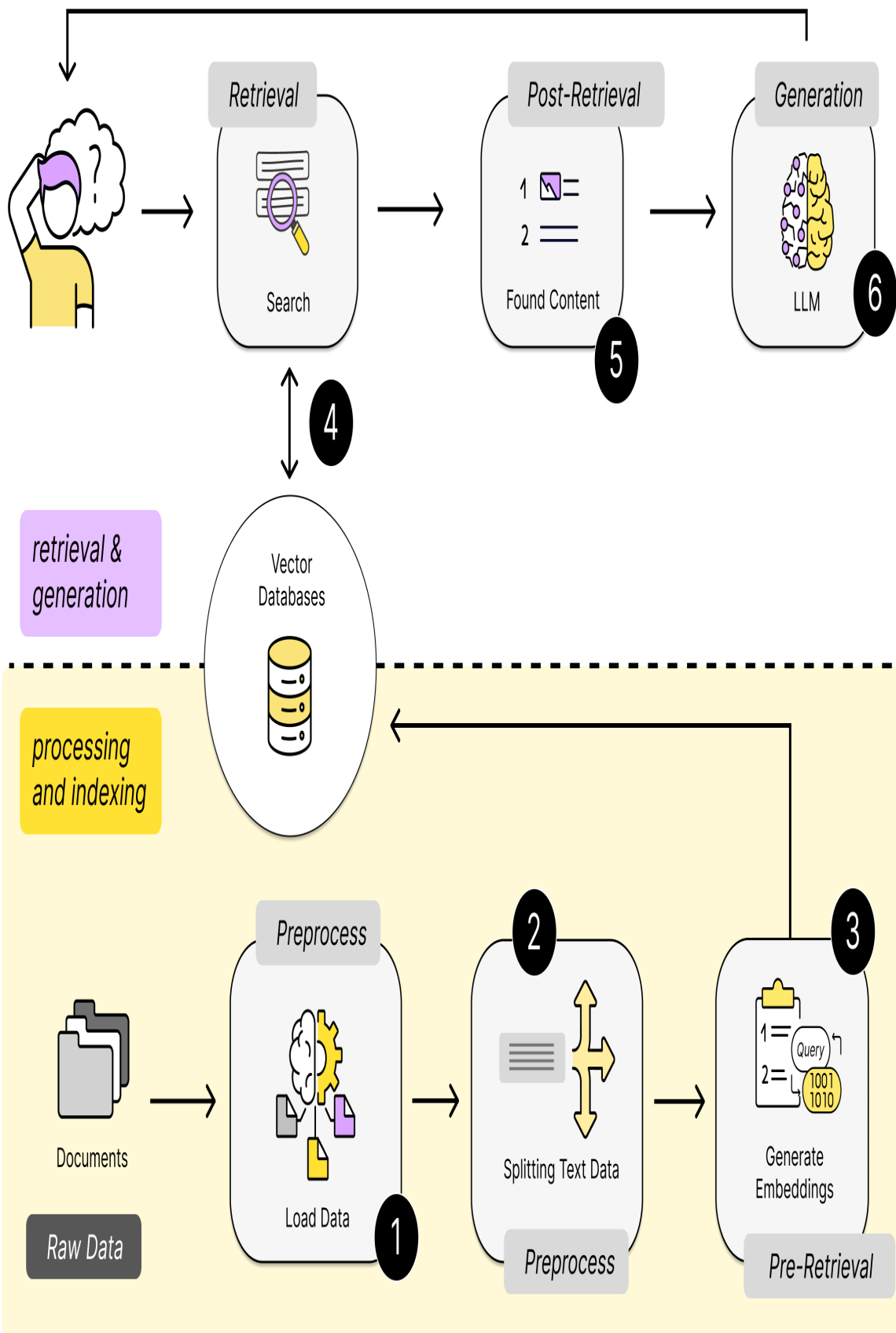


Figure 1-2. Overview of the components of a RAG system.

While the process may sound straightforward, it gets more tricky when we deal with data from different sources in different modalities. This makes the design of the loading pipelines so essential and requires creativity.

This chapter provides tools and techniques for designing customized loading pipelines.

WARNING

In this book, you will often see functions from RAG frameworks like LangChain and LlamaIndex. This makes sense because these RAG frameworks are handy and offer many functions we need to create RAG applications. Nevertheless, check first if you really need them. They are still at an early stage and constantly changing, which can be challenging when deploying apps to production. Since they are merely a collection of more established frameworks, you could also use the standalone frameworks behind LangChain and LlamaIndex.

1.1 Loading Word Files in Python

Problem

You want to load Word files (.docx, doc) into your RAG system.

Solution

You have a few options here. The most popular library is `python-docx`, which is great for creating and updating Word files. If you need to differentiate between elements like headings, paragraphs, or listings, check out `unstructured`. [Figure 1-3](#) shows the basic process. With `unstructured`, you can break down the document into its structured elements, resulting in a list of raw elements with their respective types and useful metadata.

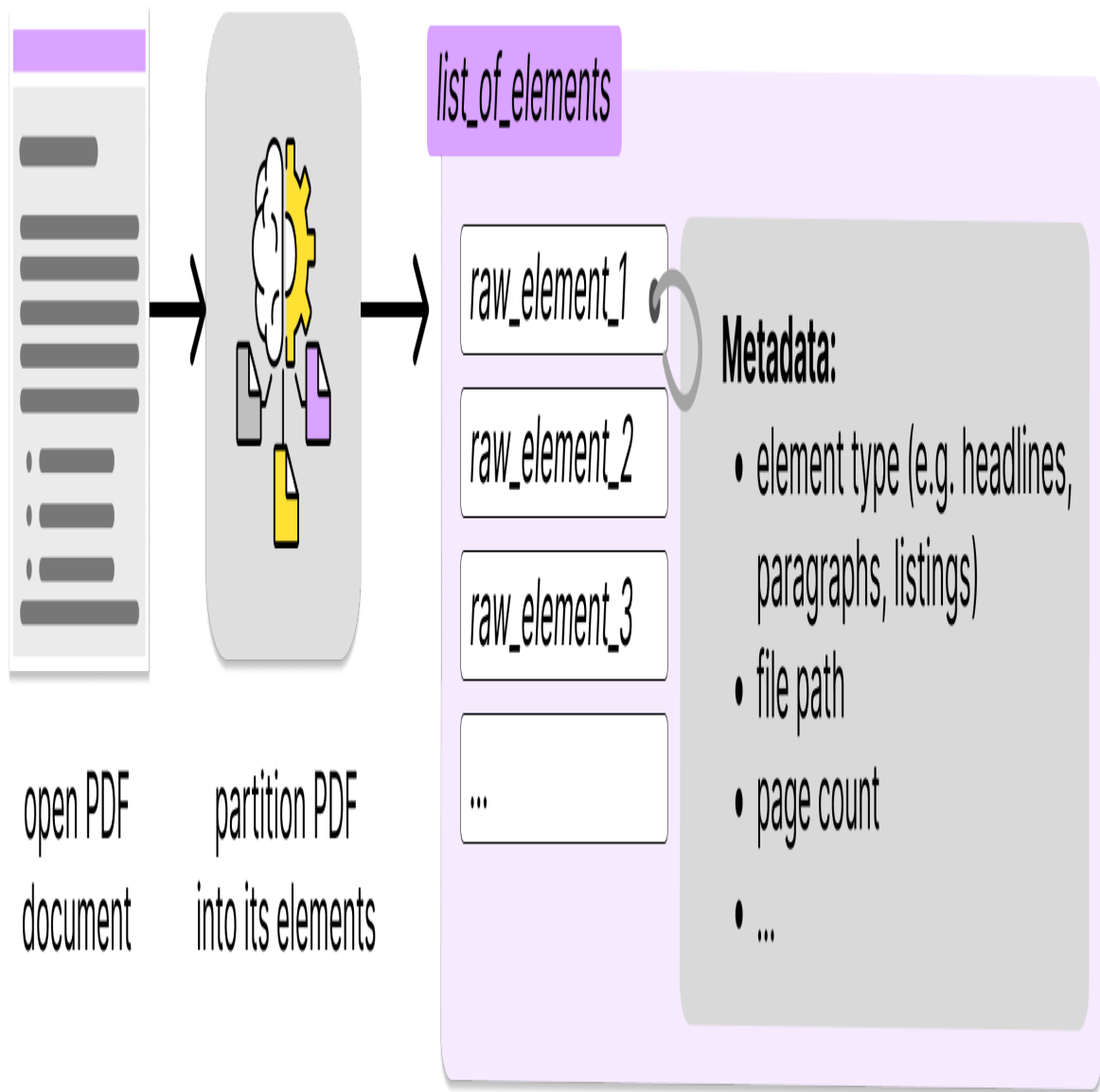


Figure 1-3. How to load Word files using python-docx or unstructured.

Let's dive into some examples using both libraries. In **Example 1-1**, we'll use the `python-docx` library to load the Word file. Then, in **Example 1-2**, we'll achieve the same result but with the `unstructured` library, which also breaks down the document into a list of elements, each with its type.

Prerequisites

In addition to `python-docx` and `unstructured` to load the PDF, we will also use `Pandas` to process the loaded data further.

Steps

In [Example 1-1](#), we are using python-docx to load the text from the sample Word file [Most_Used_Feature_Engineering_Techniques.docx](#).

To load the document and get its text as a single string, we use the `docx.Document()` function. This function returns an object with a `paragraphs` attribute, which is a list of paragraphs. In [Example 1-1](#), we iterate over the paragraphs and concatenate their `text` into one string.

Example 1-1. Load Word File using python-docx

```
import os
from docx import Document

file_path =
"../datasets/word_files/2023_Jan_7_Feature_Engineering_Techniques.d
ocx"

doc = Document(file_path)

text = ""
for paragraph in doc.paragraphs:
    text.append(paragraph.text)
```

If our RAG system needs to differentiate between elements like titles, text, and lists, we can use the `partition_docx` function. This function returns a list of objects, each with a `text` attribute for the element's text and a `type` attribute for the element's type (e.g., `Title`, `NarrativeText`, `ListItem`). In [Example 1-2](#), we iterate over these elements and create a dictionary `element_dict` for each to store the desired metadata.

Example 1-2. How to load and partition Word files using unstructured

```
from unstructured.partition.docx import partition_docx
import os
import pandas as pd

elements = partition_docx(filename=file_path)

list_of_elements = []

for element in elements:
```

```
element_dict = {
    "element_id": element.id,
    "file_path": file_path,
    "category": element.category, # e.g. "Title",
    "NarrativeText", "ListItem"
    "text": element.text,
    "last_modified": element.metadata.last_modified,
}

list_of_elements.append(element_dict)
```

The result is a list of elements. Each element is a dictionary containing the category, text, and file path.

Discussion

Word is widely used in companies to create project documentation, contracts, and books. Loading text from Word (.docx) files is straightforward. For more complex documents with listings, tables, and images, we can build customized processing pipelines for each element type. For images, we might create summaries explaining the content using LLMs, and for tables, we might summarize key insights.

Knowing the document structure can also optimize the retrieval step in an RAG system. For example, in a book with unrelated chapters, we might first find the best matching chapter title before searching for relevant information within the chapter.

See Also:

While this recipe focused on Word files, the Python library **unstructured** is especially powerful for PDFs. When your loading pipeline becomes more complex, optimizing it for one type of document can make your life easier. Instead of having multiple pipelines for different document types, you could have one pipeline optimized for PDF files, and a step before that pipeline to convert all other document types into PDFs. See **Recipe 1.10** for more on handling PDFs.

1.2 Loading PDF Files

Problem:

You want to load PDF files (.pdf) into your RAG system.

Solution

Use the **PyPDF2** library to load PDFs into your RAG system. This library allows you to extract the text from each page of the PDF. It also allows you to store the exact page number along with the text.

Figure 1-4 shows the pipeline step by step. First, it loads the PDF and then goes through each page to extract the text and store it along with some useful metadata in a dictionary. After processing all the pages, it returns a list of dictionaries, each containing the text and metadata for a page.

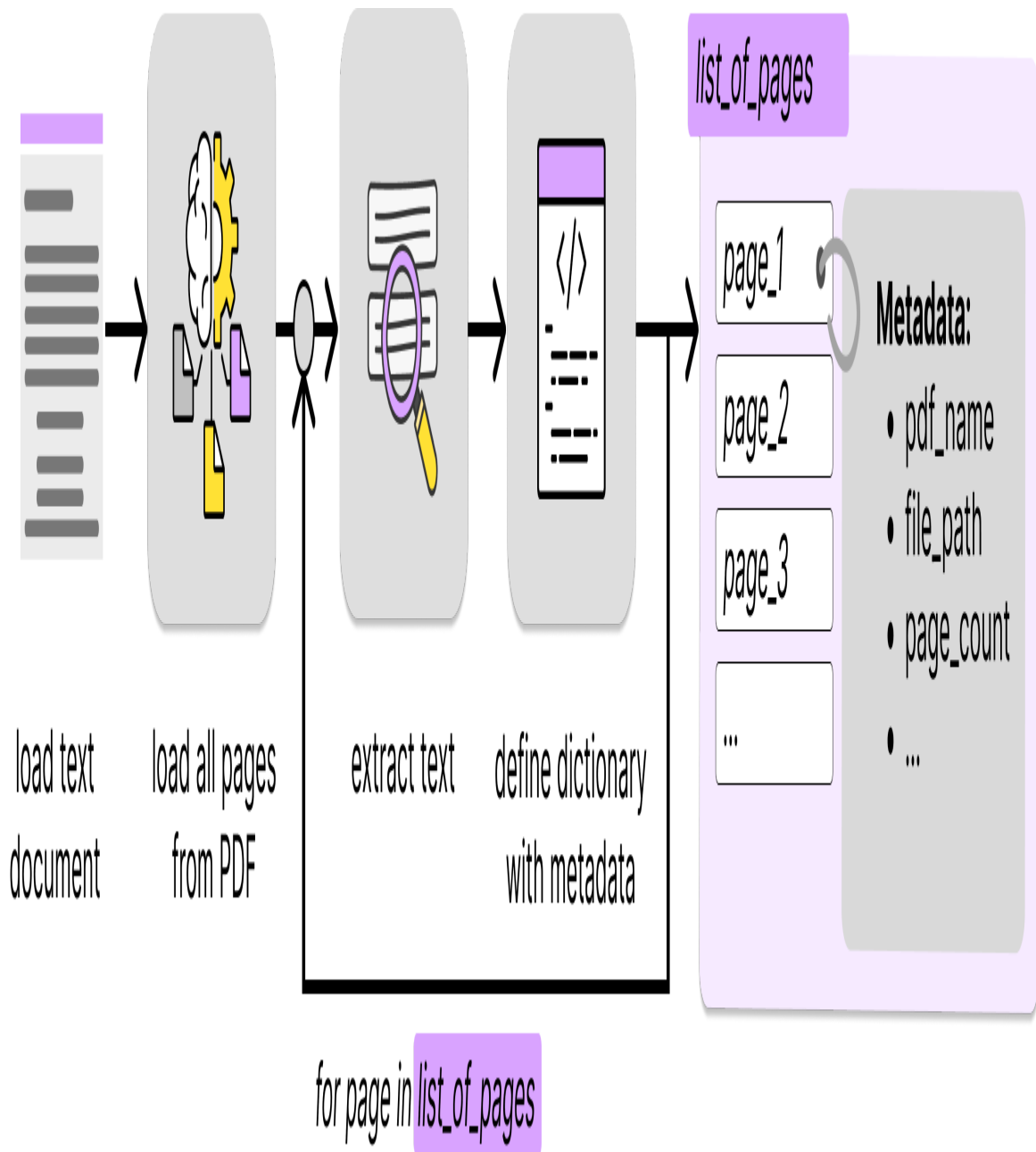


Figure 1-4. Loading PDF files and storing the text and metadata

Prerequisites

The **PyPDF2** library is an open-source library for splitting, merging, cropping, and transforming the pages of PDF files. It uses **Pillow** under the hood to extract images from the PDF.

Steps

Example 1-3 shows how to use PyPDF2 to read a PDF. The function `PyPDF2.PdfReader()` returns an object with a `pages` attribute, which is a list of dictionaries. Each dictionary contains the page's text and metadata.

We can extract the metadata we need by iterating over the pages and filling a customized dictionary for each page. In this example, the dictionary contains the page text, filename, producer, page number, and images.

Example 1-3. Loading PDF Files: Using PyPDF2 to load plain text and metadata

```
import PyPDF2
import os
import pandas as pd

file_path =
"./datasets/pdf_files/2023_Jan_7_Feature_Engineering_Techniques.docx"

with open(file_path, "rb") as file:
    reader = PyPDF2.PdfReader(file)

    # Initialize an empty string to store the extracted text
    list_of_pages = []
    page_counter = 1

    for page in reader.pages:
        page_dict = {
            "file_name": reader.metadata.get("/Title"),
            "producer": reader.metadata.get("/Producer"),
            "page_number": page_counter,
            "text": page.extract_text(),
            "images": page.images,
        }

        list_of_pages.append(page_dict)

        page_counter += 1
```

The code snippet returns a list of dictionaries, each containing the text and defined metadata (file name, producer, page number, images on the page). Later in the RAG process, the text from the page is chunked into text

chunks and translated into embedding vectors. The metadata extracted here will be linked to each text chunk, which allows us to give the correct and detailed references to the information used.

Discussion

PDF is the most commonly used file format for sharing information. Whether the original file was created in Word, PowerPoint, or another tool, it is often converted to PDF before being shared with a broader audience. As a result, most documents are available in PDF format.

Therefore, loading and processing PDF files is crucial for many RAG systems. This is evident when using RAG frameworks, as functions and tools for loading PDFs are often the most advanced and mature. That's why it can make sense to center your whole RAG application around PDF files by simply converting all other formats into PDF files. Just make sure you don't lose any important details like metadata, formatting, images, or interactive elements during the conversion.

See also:

Loading the pure text is pretty straightforward. However, it gets more challenging when we have PDFs with images and tables. You can find some techniques to handle multimodal content in PDFs in [Recipe 1.10](#).

1.3 Loading and Handling CSV and Excel Files

Problem

You want to load Excel or CSV files (.xlsx, .csv) to your RAG system.

Solution

We have three fundamentally different ways to handle Excel and CSV files in RAG systems. You can find a short introduction to each option in this chapter:

1. **“Option 1 - Transforming CSV rows into text”**: You pack the information of each row in the dataset in a text snippet. This option allows you to load the dataset like you do with pure text documents.
2. **“Option 2 - Embedding the whole table into the prompt”**: If the table is not too large, you can load the whole table into the prompt. Most models are pretty good at interpreting tables attached directly to the prompt.
3. **“Option 3 - Uploading the Excel to a SQL query and use the Text-to-SQL approach”**: If you want to perform a more complex analysis of the data, you can **save the dataset to an SQL database and use a text-to-SQL approach** to answer user questions about the inserted data.

In this recipe, you will find a short introduction to all three options. The practical example in **Example 1-4** focuses on the first option, transforming CSV rows into text.

Option 1 - Transforming CSV rows into text

You can turn each row of a CSV file into a sentence that makes sense on its own. Think of it like explaining the data to a child who has never seen it before. Take one row, look at the column headers, and describe what the data means in simple words.

Once we have these text snippets, we can treat them like regular text. Finally, we create embeddings (numerical representations) for each snippet and store them in a vector database, as shown in the image **Figure 1-5**.

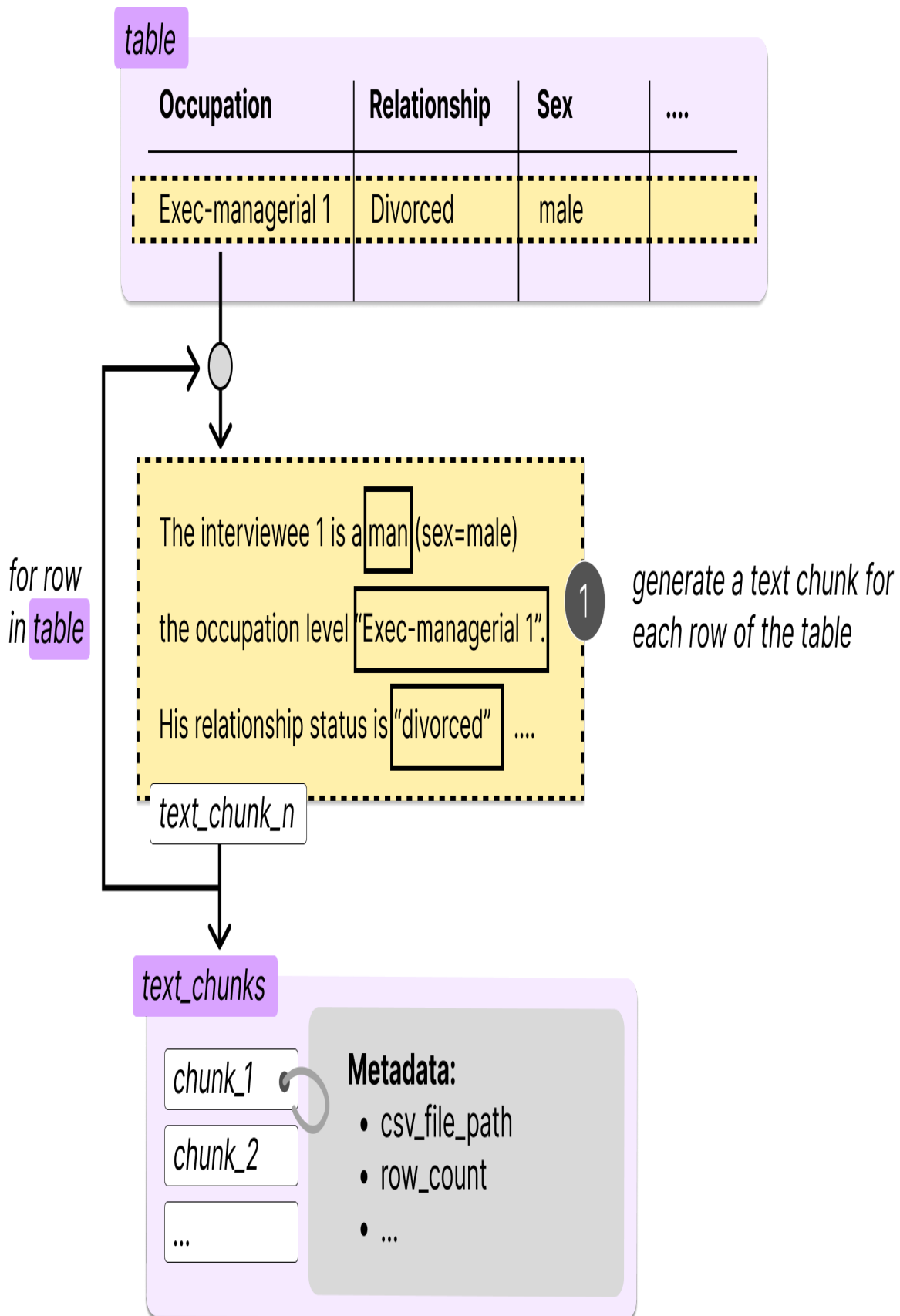
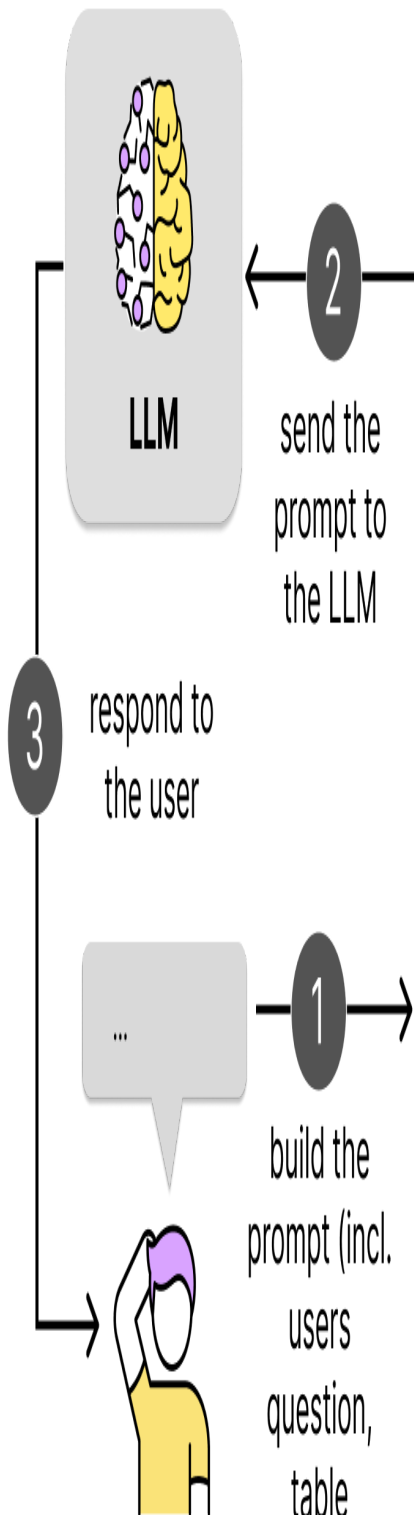


Figure 1-5. How to load CSV files by transforming CSV rows into text.

Option 2 - Embedding the whole table into the prompt

If the table is not too large, you can add the entire table to the prompt. Most models can interpret tables directly in the prompt, at least when they have the right format. Transform the table into markdown syntax to ensure the LLM recognizes it as a table.

In **Figure 1-6**, you can see a basic prompt template with an attached markdown table.



prompt

You are an assistant which helps us to interpret tabular data.

Below you can find the user's question and context section. The context section includes the data you should use to answer the user's question.

User question:

Explain the dataset attached to me. Are you seeing any pattern in the data?

Context:

Name	Age	Occupation
Alice	30	Software Engineer
Bob	25	Data Analyst

Figure 1-6. How to load CSV files by embedding the whole table into the prompt.

Option 1 is straightforward to implement but can only answer questions using information from a few rows. Option 2 can be expensive for large tables and struggles with complex queries. For a more detailed analysis, use the text-to-SQL approach in option 3.

Option 3 - Uploading the Excel to a SQL query and use the Text-to-SQL approach

For more complex queries, like aggregations over the whole table, we can upload the Excel file to an SQL database and use the text-to-SQL approach. The LLM will generate an SQL query to get the needed information from the database based on the user's question.

Once the data is retrieved, we create a prompt with the user's question and the data (see [Figure 1-7](#)).

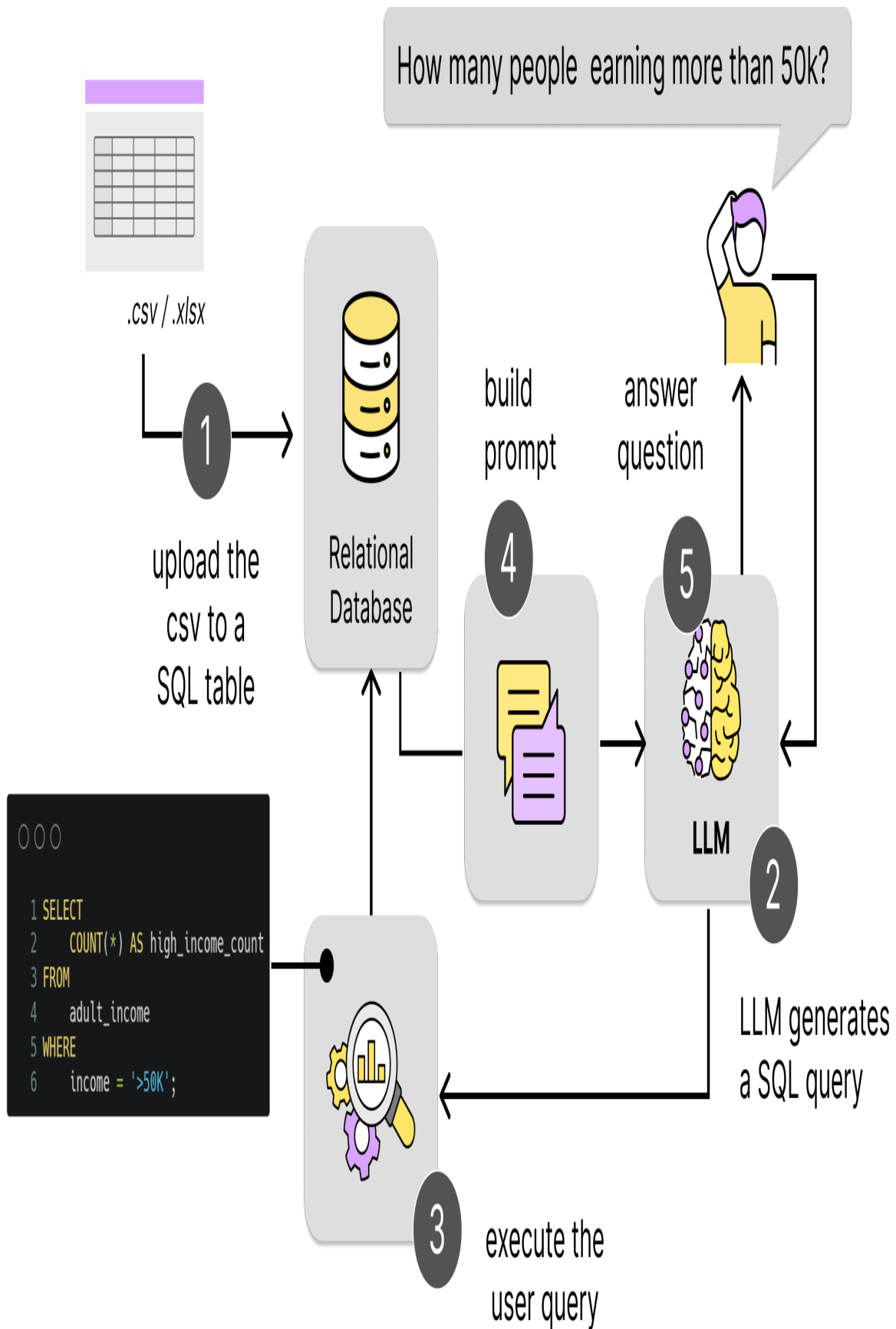


Figure 1-7. How to load CSV files by uploading the Excel file to an SQL database and using the Text-to-SQL approach to answer user questions about the inserted data.

The rest of this recipe shows a practical example for option 1. [Example 1-4](#) is transforming each line of the loaded rows into a text chunk.

Prerequisites

We will use the open-source [census income dataset](#), which is commonly used for binary classification tasks. The dataset will be saved in an Excel file containing 15 columns and 48,000 rows.

To load the Excel file, we will use the Python library [openpyxl](#). For further processing of the tabular data, we will use [Pandas](#).

Steps

First, we load the Excel file `census-income.xlsx` into a [Pandas](#) data frame. Once loaded, [Example 1-4](#) applies the function `create_text_description` to each row of the data frame and thus creates a new column `text_description` with a text snippet explaining the values in the other columns. To create the text chunk, we are using a static template, which you can find in [Example 1-4](#).

Example 1-4. How to transform CSV rows into text

```
import os
import pandas as pd

file_path = "../datasets/csv_files/census-income.xlsx"
df_excel = pd.read_excel(io=file_path)

def create_text_description_of_row(row):
    row["text_description"] = (
        f"""The candidate {row['age']} years old is working in the
            {row['workclass']} sector. The candidate was born in
            {row['native-country']}, is {row['marital-status']}
            "and has a {row['relationship']} relationship.
            "The candidate has a {row['education']} degree
            and is working as a {row['occupation']}.
            The income of the candidate is {row['income']} and he
            is from."""
```

```
)
```

```
return row
```

```
# Apply the function create_text_description_of_row to each row of  
the data frame  
df_extended = df_excel.apply(create_text_description_of_row,  
axis=1)
```

Discussion

The challenge for RAG systems is that LLMs and embedding models are designed to work with text. This recipe showed some methods to prepare tables for use with LLMs. It's important to understand the strengths and limitations of each approach.

- For small tables, convert them to Markdown format. This helps the model recognize it as a table and include it in your prompt.
- For large Excel files, break down the information and describe each row as a separate piece of data. This method is useful for simple questions like *Is candidate 25 earning more than 50k?* or *What is the Education Level of candidate 64?*.
- For complex questions, like *What percentage of candidates with a Bachelor's degree earn more than 50k?*, use a method that can handle aggregations on the entire dataset, such as the Text-to-SQL approach. Libraries like Vanna AI make this straightforward to implement.

See also

In this recipe, we converted each row of the dataset into a text snippet to use with text embedding models. We can use the same approach also for classification tasks. For more details, refer to the paper [TabLLM: Few-shot Classification of Tabular Data with Large Language Models](#).

1.4 Querying a PostgreSQL Database

Problem

You want to connect your RAG system to a PostgreSQL database to pull data from it.

Solution

Prerequisites

First, install the PostgreSQL server and **pgAdmin** on your machine. PgAdmin is a popular tool for managing PostgreSQL databases. While you can run all SQL queries directly from Python, pgAdmin makes it easier to browse data and execute queries manually. You can find installation instructions for PostgreSQL at [postgresql.org/download/](https://www.postgresql.org/download/). PgAdmin will be installed by default when you install PostgreSQL.

If you work with multiple database types, consider using **DBeaver** or **DataGrip**. These tools support various databases like MySQL, PostgreSQL, SQLite, Oracle, DB2, and SQL Server.

To connect to and load data from our databases, use **SQLAlchemy**. SQLAlchemy is the most used Python SQL toolkit. It gives us the power and flexibility of SQL in Python.

Steps

When you launch pgAdmin for the first time, you can either connect to an existing PostgreSQL server or create a new local server. Follow the steps below to populate and query the database:

1. **Replicate Sample Database:** To have some data to work with, we will replicate the sample database from [w3schools.com](https://www.w3schools.com). Follow the tutorial on [w3schools.com](https://www.w3schools.com) to fill the database with sample data.
2. **Install Required Libraries:** Next, install the Python libraries **SQLAlchemy** and **psycopg2-binary** to connect to the database.

3. **Create User and Grant Permissions:** Create a user and grant the necessary permissions by following the PostgreSQL documentation on [creating a user](#) and [granting permissions](#). Store the user credentials in a .env file.
4. **Set Up Connection and Execute Query:** With the user created, set up a connection. In [Example 1-5](#), we use this connection to execute an SQL query that retrieves all data from the table `categories`.

Example 1-5. Loading Data from PostgreSQL: Connect to the database and execute queries using SQLAlchemy

```
import os
import pandas as pd
from sqlalchemy import create_engine

connection_string = (
    f"postgresql+psycopg2://{username}:{password}@{host}:"
    f"{port}/{database}"
)
engine = create_engine(connection_string)

with engine.connect() as connection:
    query = """SELECT * FROM categories ORDER BY category_id ASC
    """
    result = pd.read_sql(query, connection)
    print(result)
```

After running the query, the retrieved data will be stored in the data frame `results_df`. The code snippet fetches all records from the table `categories` and sorts them by the column `category_id`.

Discussion

Most of our data is stored in relational databases, and PostgreSQL is one of the most popular ones. Websites, tools, and machines collect and save data in these databases.

When creating a web app for our RAG workflows, we need to load data from or save data to our database. PostgreSQL is a great choice because it

can store text data and, with the *pgvector* extension, it can also store vector embeddings and perform similarity searches. To learn how to generate embeddings, store them in PostgreSQL, and calculate distance metrics, see [Link to Come].

See also

In this recipe, we focused on loading data from PostgreSQL. In other recipes, you will find different parts in our RAG system using PostgreSQL:

- If you want to know how to store embeddings in PostgreSQL and perform similarity searches on top of it, you can find it in [Link to Come]
- [Link to Come] will use PostgreSQL as an app database for RAG web apps

1.5 Loading Audio Files by Using Speech-to-Text Models

Problem

You want to load audio files to your RAG system.

Solution

To load audio files to our RAG system, we first need to convert them into text. We achieve this using speech-to-text models. These advanced models are trained on thousands of hours of audio in multiple languages and have significantly improved in recent years. They can automatically detect languages and handle loud background noise effectively.

Prerequisites

I will use OpenAI's Whisper model for this task. You can either install Whisper locally or use it through OpenAI's API. The choice depends on your use case:

- For quick setup and scalability, use a cloud service like AWS, Azure, GCP, or another platform that offers text-to-speech models.
- If you need to handle sensitive data, you can host an open-source model like Whisper on your own servers. This ensures that only you have access to the data.

In **Example 1-6**, we will use OpenAI's cloud services. We will load the OpenAI key from a .env file using the **python-dotenv** library.

Steps

Example 1-6 uses the official OpenAI Python library. Just provide the `file_path` of the audio file and the model you want to use (in this case, *whisper-1*).

That's all it takes. One API call and your product can be labeled as *AI-powered*.

Example 1-6. How to load and transcribe audio files using Whisper

```
import os
import openai

audio_file_path = "../datasets/audio_files/harvard.wav"

# initialize the OpenAI client with your API key
client = openai.OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

with open(audio_file_path, "rb") as audio_file:
    transcription = client.audio.transcriptions.create(
        model="whisper-1", file=audio_file
    )
```

See also

If you need to transcribe not just audio files but also video files, refer to [Recipe 1.11](#). This involves breaking the video into parts and processing each part separately, with the audio track being a key element.

For a detailed tutorial on using Whisper, check out the guide]from OpenAI on [HuggingFace](#).

1.6 Extracting Text from Images and PDFs Using OCR

Problem

You want to extract text from images or image-like documents using open-source OCR engines you can host on your own infrastructure.

Solution

This recipe explains how to extract text using *Optical Character Recognition (OCR)* engines. You can also use multimodal foundation models for this task (see [Recipe 1.7](#)). Each method has its pros and cons. Multimodal models are flexible and can be easily adjusted by changing the prompt. OCR engines are fast, cost-effective, and can be run on your own servers.

[Figure 1-8](#) illustrates how OCR engines extract text from images, including scanned PDFs. For scanned PDFs, the process involves loading the PDF, saving each page as an image, and then using the OCR engine to extract text from each image. The end result is a list of dictionaries, each containing the image path and metadata for a page.

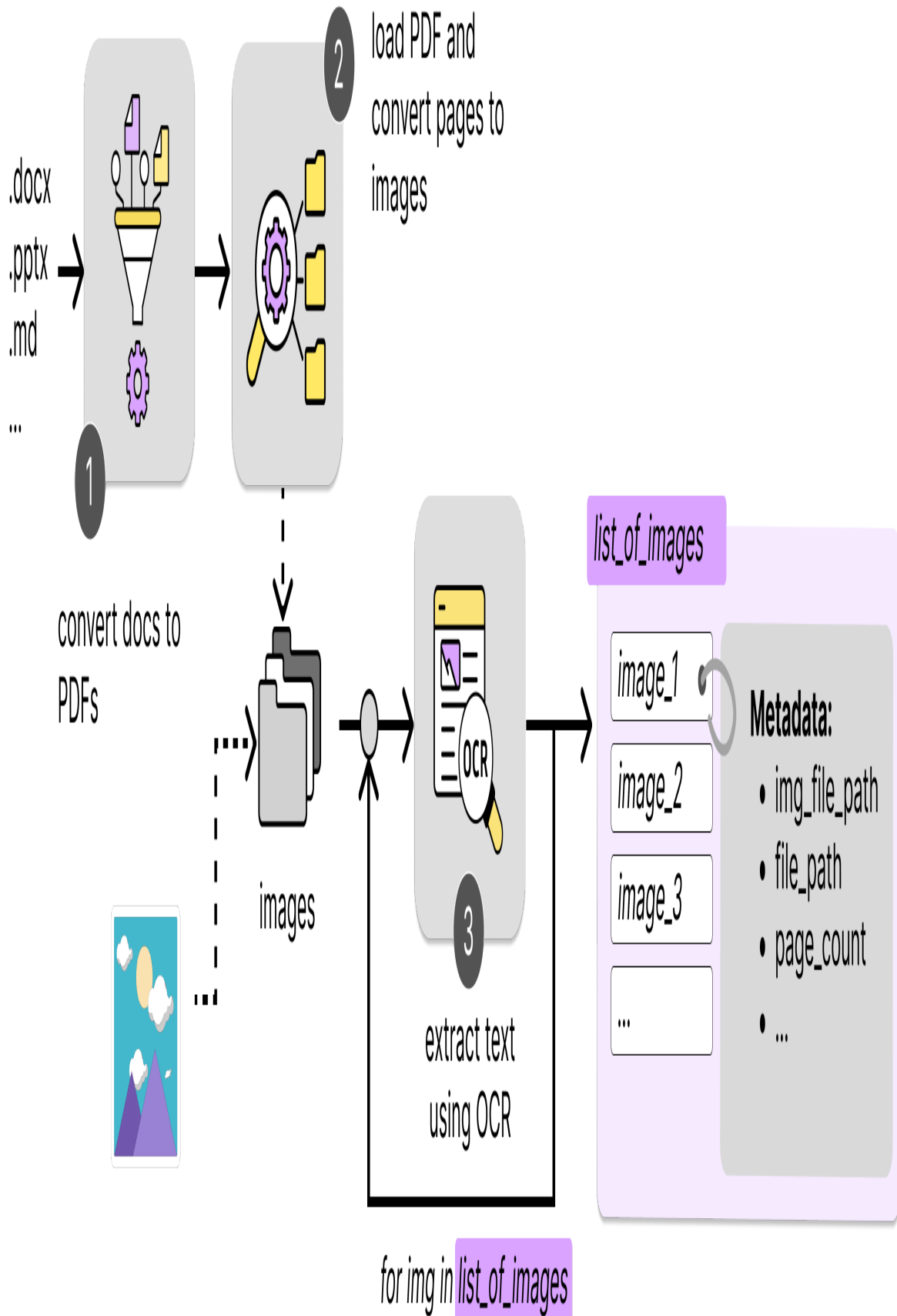


Figure 1-8. OCR engines can extract text from images, e.g., from photographs, technical drawings, or scanned contracts.

Prerequisites

The most popular OCR engine is **Tesseract**. You can either install **Tesseract** locally or use cloud services like Google Cloud Vision, Amazon Textract, or Microsoft Azure AI Vision.

For **Example 1-7**, we will install **Tesseract** locally. You can find installation guides for different operating systems in the **Tesseract documentation**. If you are on Windows, you can use the **Windows installer** provided by the University of Mannheim.

Once Tesseract is installed, we will install the required Python packages: **pdf2image**, **pytesseract**, and **pillow**.

Steps

Let's go through two examples. In **Example 1-7**, you'll see how to extract text from a sample finance PowerPoint slide, which you can find in the **ook's repo**. In **Example 1-8**, you'll learn how to extract text from a PDF by first converting each page of the PDF into an image.

Example 1-7. Example 1: Loading text from a .png image using Tesseract

```
import os
from pdf2image import convert_from_path
from PIL import Image
import pytesseract

# Load the sample .png file
image =
Image.open(file_path="../../datasets/images/example_finance_reporting_
slide.png")

# Use Tesseract to do OCR on the image
text = pytesseract.image_to_string(image)
```

Example 1-8 shows an example for a PDF file. The code snippet loads the PDF *Most_Used_Feature_Engineering_Techniques.pdf* as an image and then use Tesseract to extract the text from each PDF page.

Example 1-8. Example 2: Loading text from PDF files using Tesseract

```
import os
from pdf2image import convert_from_path
from PIL import Image
import pytesseract

file_path =
"./datasets/images/Most_Used_Feature_Engineering_Techniques.png"

# Convert PDF to a list of images
images = convert_from_path(file_path=file_path)

text = []
for i, image in enumerate(images):
    page_text = pytesseract.image_to_string(image)
    text.append(page_text)
```

Discussion

Extracting text alone isn't very useful for photos of people, landscapes, or objects. OCR engines aren't suitable for these types of images.

However, OCR engines are effective for images that primarily contain text, such as scanned documents. They offer a reliable and cost-effective method for extracting text from these images. When converting PDF pages into images, make sure to save the file paths for both the PDF and the images, as well as the page numbers.

I rarely use OCR models these days because multimodal models make text extraction so straightforward. The only time I would choose OCR models is when I know the documents are mostly text, have a consistent structure, and need to be processed quickly and cost-effectively.

See also

With multimodal models, we can take it a step further. Depending on the specific images, we can either ask the model to extract just the text (see [Recipe 1.7](#)) or to describe every detail of the image (see [Recipe 1.8](#)). The models can even decide on their own what makes the most sense for each image.

1.7 Extracting Text from Images using Multimodal Models

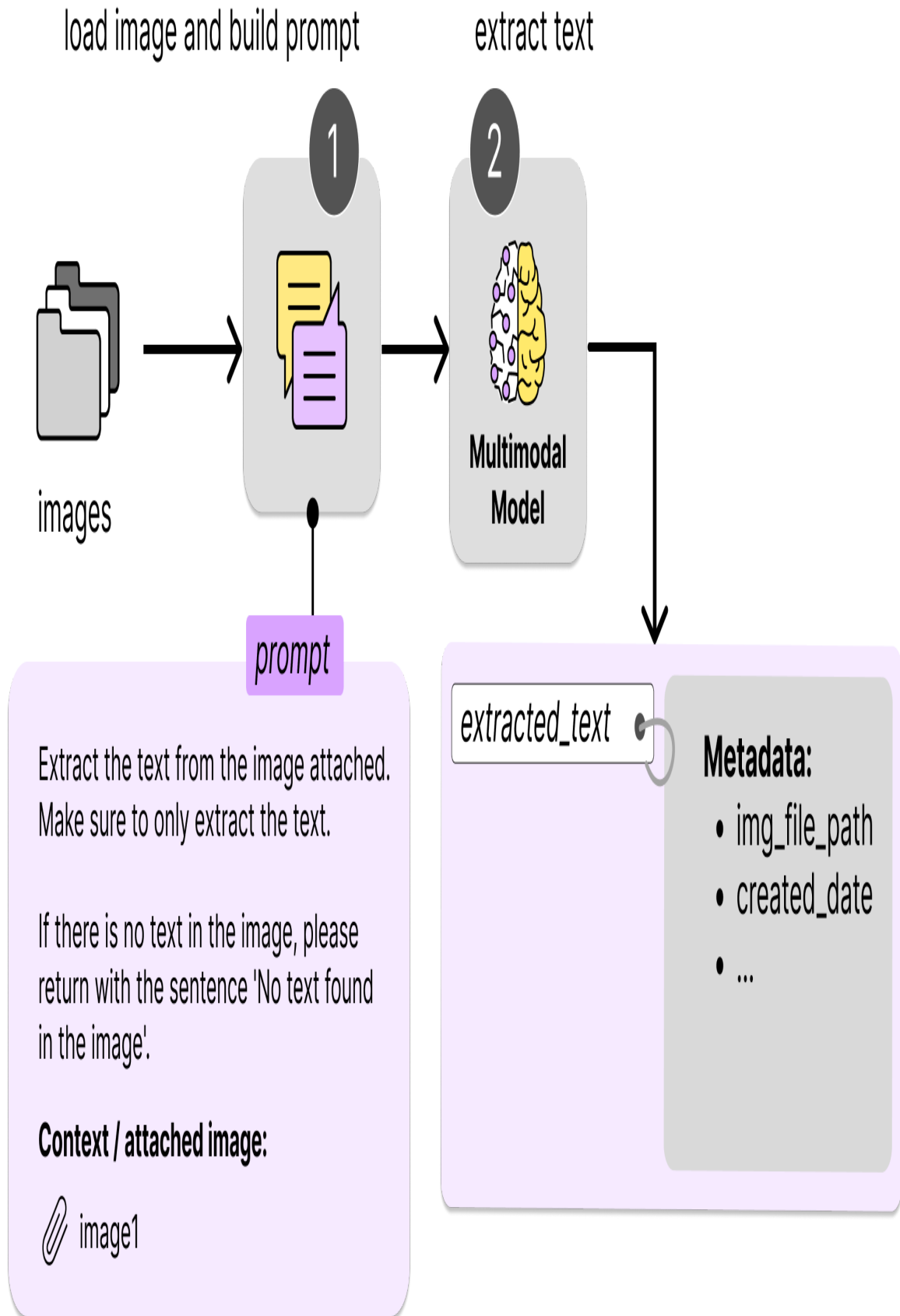
Problem

You want to extract text from images or image-like documents in a robust and highly flexible way.

Solution

Recipe 1.6 used OCR engines to extract text from images. In this recipe, we use multimodal models.

The concept is simple. We create a basic prompt and ask the model to describe the image in detail. In **Figure 1-9**, you can see an example of a prompt and the model's output. For each image, we aim to get a dictionary containing the extracted text and relevant metadata, such as the image path or creation date.




```

{base64_image}",
    },
],
},
],
max_tokens=500,
)

content = response.choices[0].message.content

```

See also

The concept is straightforward and powerful. In [Example 1-9](#), we used a multimodal model to extract text from images. What makes these models so powerful is their flexibility; by simply adjusting the prompt, we can use them for various preprocessing tasks. Here are some examples from this chapter:

- [Recipe 1.8](#) shows how to use multimodal models to extract text and create detailed summaries for images.
- [Recipe 1.9](#) demonstrates how to use multimodal models to extract key insights from tables and generate text summaries.

1.8 Generating Text Summaries for Images Using Multimodal Models

Problem

You want to summarize images to capture their key information. This allows us to use text embedding models to create embeddings that represent the image content.

Solution

There are two ways to create embeddings for images:

1. Directly create embeddings for the images using an image embedding model.
2. Generate text summaries for the images using multimodal models, then create embeddings for those summaries.

Figure 1-10 illustrates the second approach, which is used in this recipe. This method is generally more practical and flexible, as it can handle various types of images, whether they are photos of people, landscapes, or technical charts.

load image and build prompt

extract text

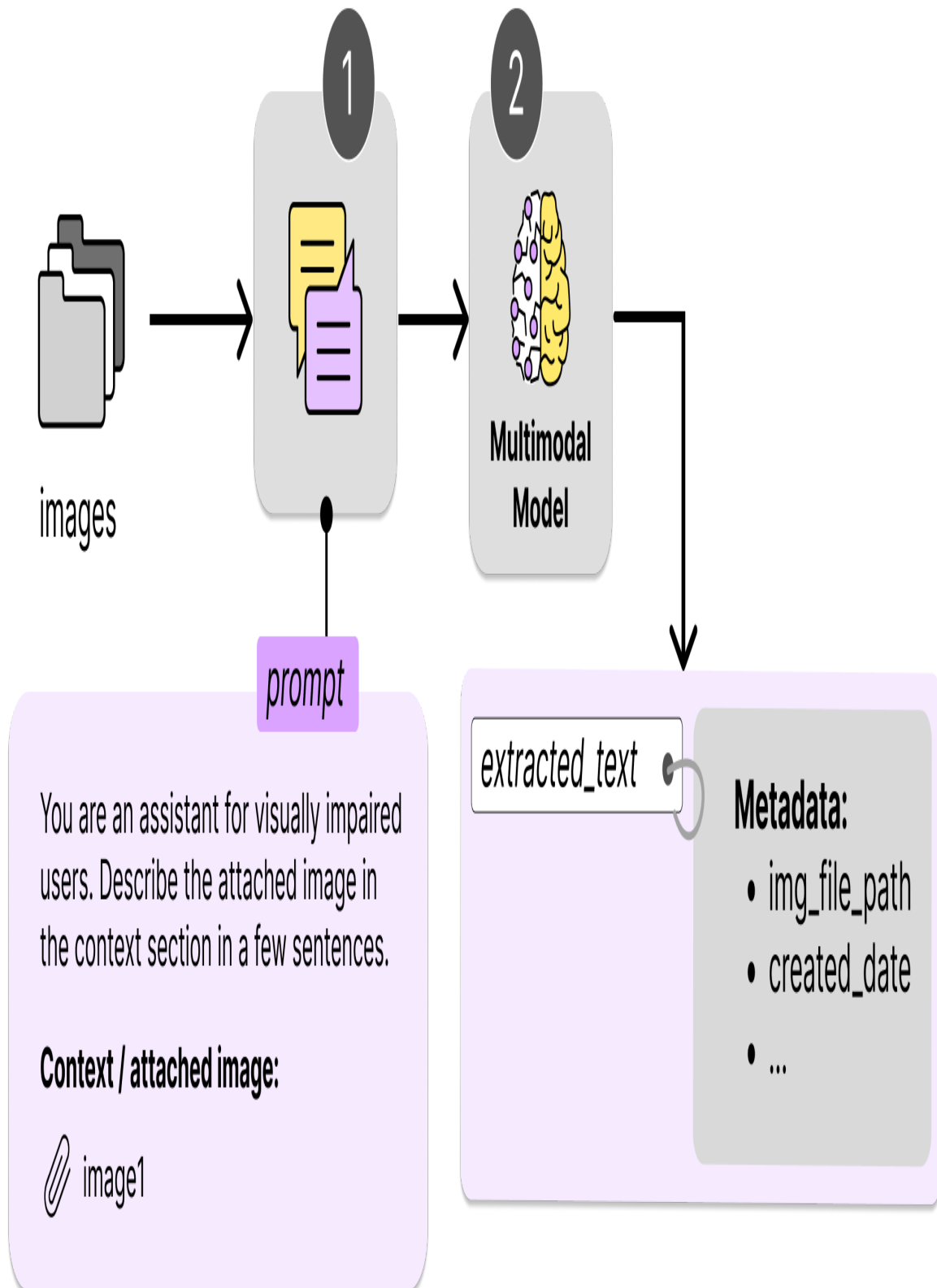


Figure 1-10. Generating text summaries for images using multimodal models.

Prerequisites

We need a multimodal model that can understand and analyze images. In [Example 1-10](#), we use GPT-4o from OpenAI. Other good options include [Anthropic's 3.5 Sonnet](#) or [Google's Gemini 1.5](#).

Since we are using OpenAI models, we will use the `openai` Python library. First, generate and store an API key in your OpenAI account. Avoid hard-coding the API keys; instead, store them in a `.env` file and load them using `python-dotenv`.

Steps

[Example 1-10](#) shows how to load and analyze a photo of Vietnam's capital city. We then give the model simple instructions on how to describe the image in detail.

Example 1-10. Generate Text Summaries for Images: Load the image and generate text summaries

```
import base64
import openai

image_path = "../datasets/images/vietnam.png"

with open(image_path, "rb") as image_file:
    base64_image = base64.b64encode(image_file.read()).decode("utf-8")

    prompt = (
        "You are an assistant for visually impaired users. "
        "Describe the image in detail."
    )

    response = openai.chat.completions.create(
        model="gpt-4o",
        messages=[
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {
```

```

        "type": "image_url",
        "image_url": {
            "url": f"data:image/jpeg;base64,
{base64_image}",
        },
    ],
    max_tokens=150,
)

content = response.choices[0].message.content

```

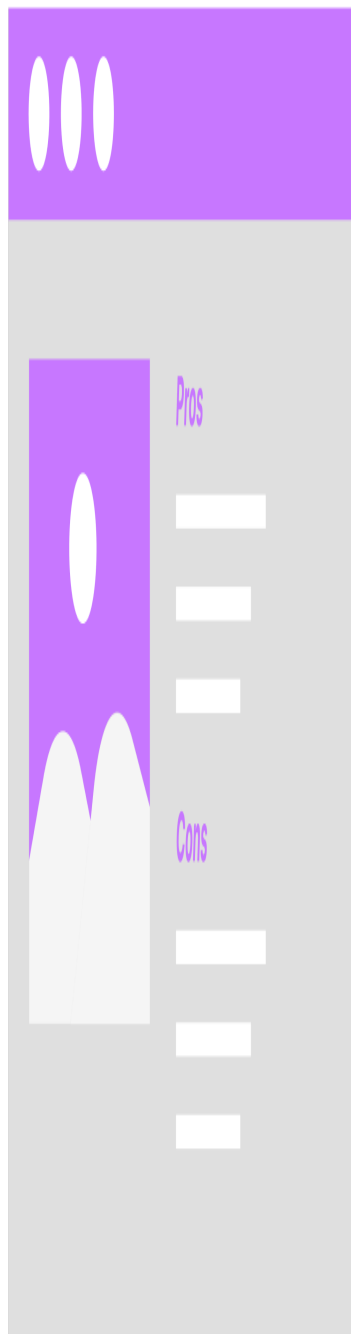
Discussion

With the rise of multimodal models, many older technologies are becoming outdated. Tasks that used to require specialized NLP or computer vision models can now be handled by multimodal models with the right prompt.

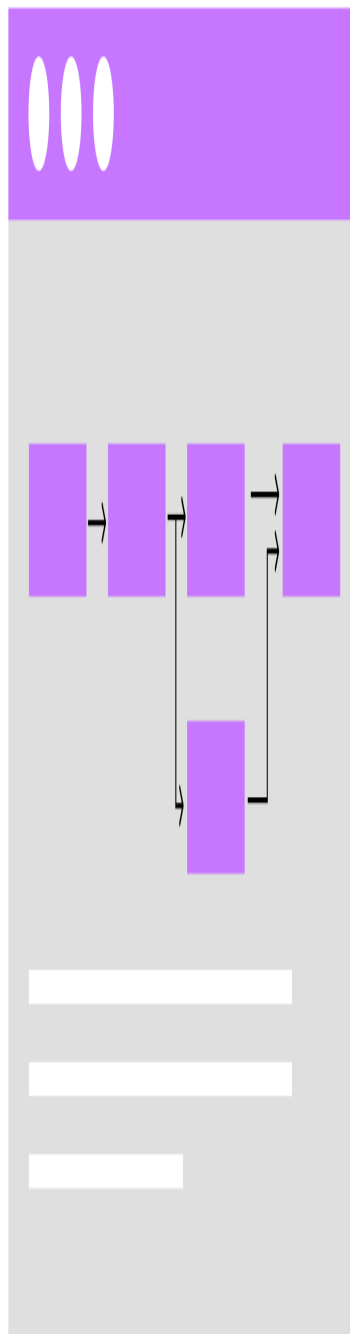
However, there are trade-offs:

- Smaller, specialized models like OCR engines are **fast and cost-effective**. They can be hosted locally, which is great for sensitive data.
- Multimodal models are **powerful and versatile**. They can handle complex images, extract text, or describe details. These models are **flexible** and can adapt their output based on the image type. For example, they might focus on facial expressions in a photo of a person or on process steps in a technical chart. **Figure 1-11** shows different types of PowerPoint slides. Depending on the type of slide, different details are important to understand the chart.

Text-based slides



Schematics / Process Flows



Data Report

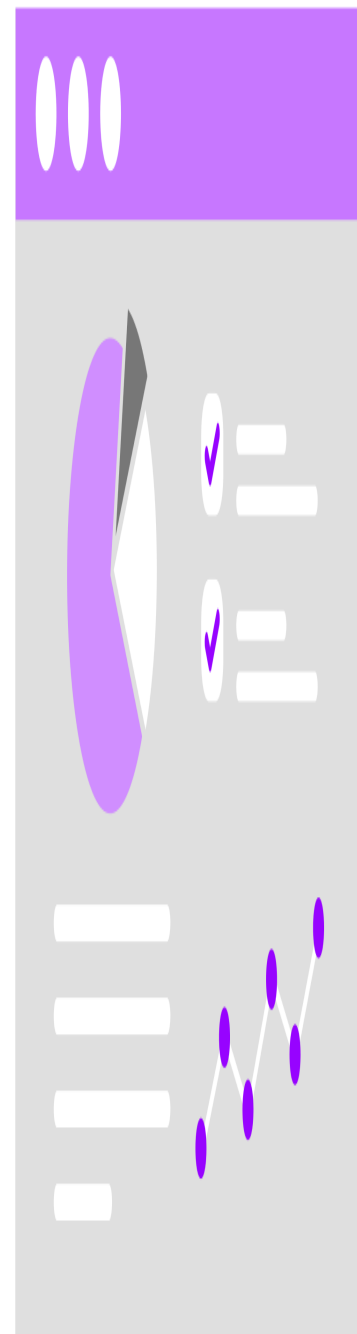


Figure 1-11. Extracting text from different image types

What's best for your use case depends mainly on the types of images and the amount of data you need to process. I recommend using a multimodal model when handling only a few hundred images.

See Also

Multimodal models are versatile because they are not designed for a specific use case but aim to replicate the human ability to generate new text. They have developed this skill by training on a large text corpus, providing them with a vast range of knowledge.

By asking the right questions, we can use those capabilities for a wide range of data processing tasks, including simple classification and clustering.

Here are some examples from the book:

- **Recipe 1.9:** The model generates text summaries for tables.
- **Recipe 2.3:** The model reads a text snippet and creates hypothetical questions based on the information.
- The most advanced use is the agentic RAG concept, which mimics human problem-solving by iteratively refining the approach (see [Link to Come]).

TIP

In the recipe, we generated summaries for images. Once generated, we can create embeddings for them using text embedding models. This process converts the image information into a vector format.

Alternatively, we can directly create embeddings for the images using models that bridge the gap between image and text content, such as OpenAI's CLIP (Contrastive Language-Image Pre-training). For more details, see [Link to Come], which explains how to use CLIP for a multimodal RAG application.

1.9 Generating Text Summaries for Embedded Tables Using Multimodal Models

Problem

You want to extract insights from tables and translate them into text summaries. This allows us to generate embeddings using a text embedding model.

Solution

In our RAG system, we store content as disconnected pieces of information. If we only extract the text from tables and store it, the embedding models might miss the key insights when creating vector embeddings.

To solve this, we can use LLMs to generate text summaries of the tables. These summaries help the embedding models capture the main points more effectively. [Figure 1-12](#) shows this process.

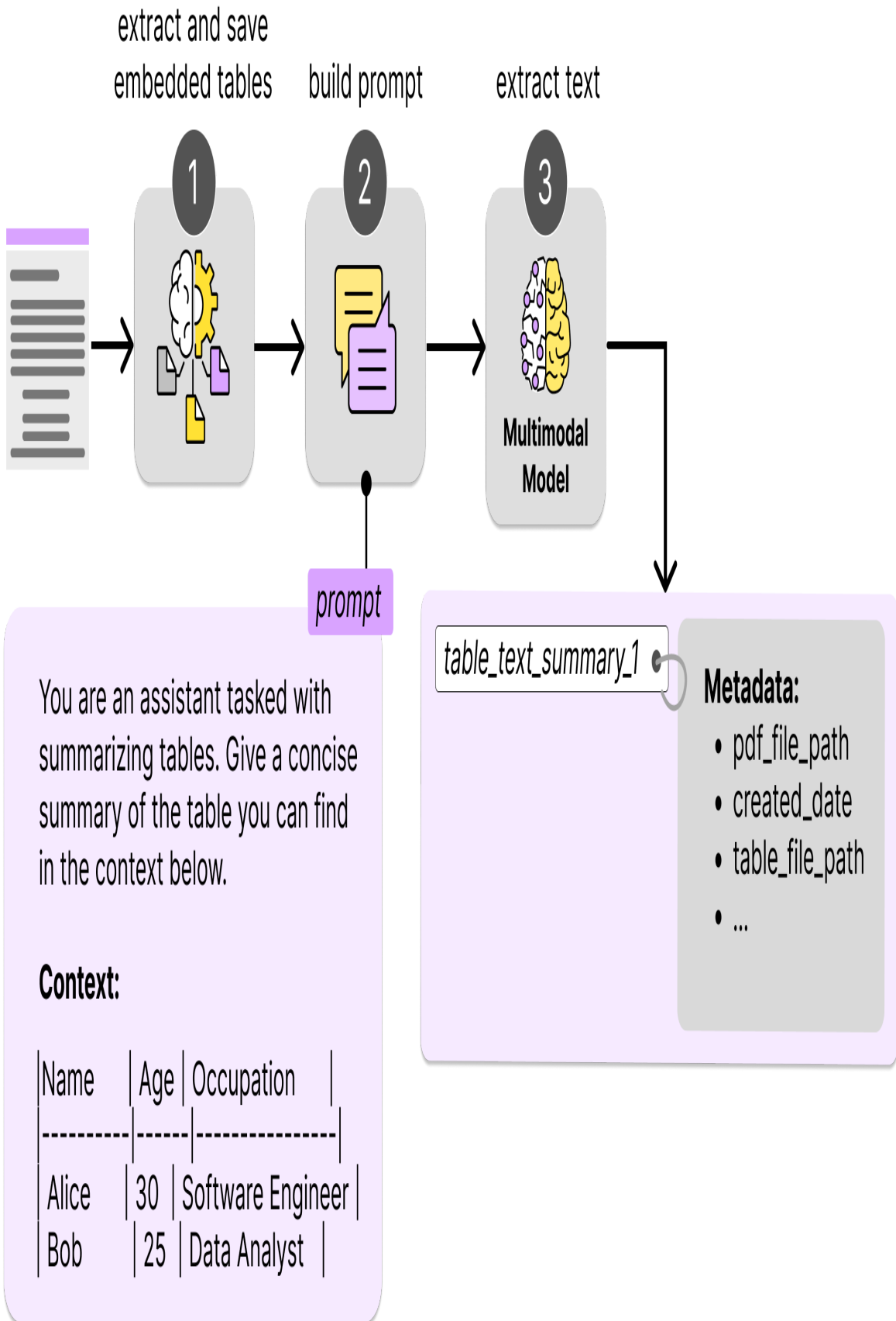


Figure 1-12. How to generate text summaries for tables using multimodal models.

Prerequisites

We use the `unstructured` library to break down the PDF into text, tables, and images. This library relies on `PDFMiner`, a tool for extracting text from PDF files. For processing, we use OpenAI's GPT-4o model through the OpenAI Python library.

Steps

Example 1-11 uses `unstructured` to partition the PDF and extract the text and tables. We will save the extracted tables in a Python list called `tables`.

Example 1-11. How to generate text summaries for tables: Partition the PDF into text and tables

```
import os
from unstructured.partition.pdf import partition_pdf

pdf_file_path = "../datasets/pdf_files/adult_data_article.pdf"

tables = []
texts = []

# partition the PDF file into its elements
raw_pdf_elements = partition_pdf(
    filename=pdf_file_path,
    strategy="hi_res",
)

for element in raw_pdf_elements:
    if "unstructured.documents.elements.Table" in
str(type(element)):
        tables.append(str(element))
```

The next step is to summarize the table using a suitable LLM.

In **Example 1-12**, we will use OpenAI's GPT-4o. The prompt is a simple question asking the LLM to highlight the key points of the table. You can customize this prompt to better fit your specific needs. Give it a try.

Example 1-12. How to Generate Text Summaries for Tables

```
from openai import OpenAI
import pandas as pd

def summarize_tables(row):
    summary_prompt = f"""You are an assistant tasked with
    summarizing tables. \
    Give a concise summary of the table. Table
    chunk: {row.table}"""

    # Initialize the OpenAI API client and generate the table
    # summary
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": summary_prompt}],
        temperature=0.7,
        max_tokens=150,
    )

    row["table_summary"] = response.choices[0].message.content

    return row

# create a pandas dataframe from the tables
tables_df = pd.DataFrame(tables, columns=["table"])

# add a column to the dataframe to store the summaries
tables_df = tables_df.apply(summarize_tables, axis=1)
```

Discussion

The LLM describes the table in a way that makes it interpretable as a standalone piece of information, making it suitable for our text embedding models.

When storing the summary and creating embeddings, we need to store the link to the original table. When the retrieval step of our RAG system identifies the key insights from the table as relevant to answering a user's question, it makes sense to include the whole table instead of just the

summary in the final prompt. The LLM should be capable of analyzing the raw table. This way, we don't lose any information along the process.

See also

Today, we use LLMs and multimodal models for many data processing tasks. These models can interpret data, rephrase text, summarize images or text, and even make decisions. We can use those models in countless use cases, often just requiring a different prompt. Here are some examples from this book:

- [Recipe 1.8](#) shows how to use multimodal models to create text summaries for images.
- [Recipe 2.3](#) demonstrates how to show the model a text snippet and ask it to generate hypothetical questions based on the information.
- The most advanced application is the agentic RAG concept, which mimics human problem-solving by iteratively refining the approach (see [\[Link to Come\]](#)).

1.10 Parsing PDFs with Multiple Media Content Using Unstructured and Multimodal Models

Problem

You want to load a PDF with multimodal content, such as images and embedded tables, into your RAG system.

Solution

Multimodal models open new doors to process and analyze documents with images and tables.

Earlier in the book, we saw how to generate text summaries for images in [Recipe 1.8](#) and how to generate text summaries for tables in [Recipe 1.9](#). In this recipe, we use both concepts to interpret the different modalities of a PDF document.

[Figure 1-13](#) shows the full process of loading a PDF with multimodal content. It requires a multi-step pipeline with the following steps:

1. **Partition Data:** Load raw data and partition it into text, images, and tables
2. **Generate Summaries:** Use a multimodal model (such as [OpenAI's GPT-4V](#), [Anthropic's 3.5 Sonnet](#) or [Google's Gemini 1.5](#)) to generate text summaries for the images and tables
3. **Create Embeddings:** Generate text embeddings for the text chunks, image summaries, and table summaries
4. **Store Embeddings:** Add the embeddings to the vector store. Store the metadata, like timestamps, file paths, page numbers, etc.

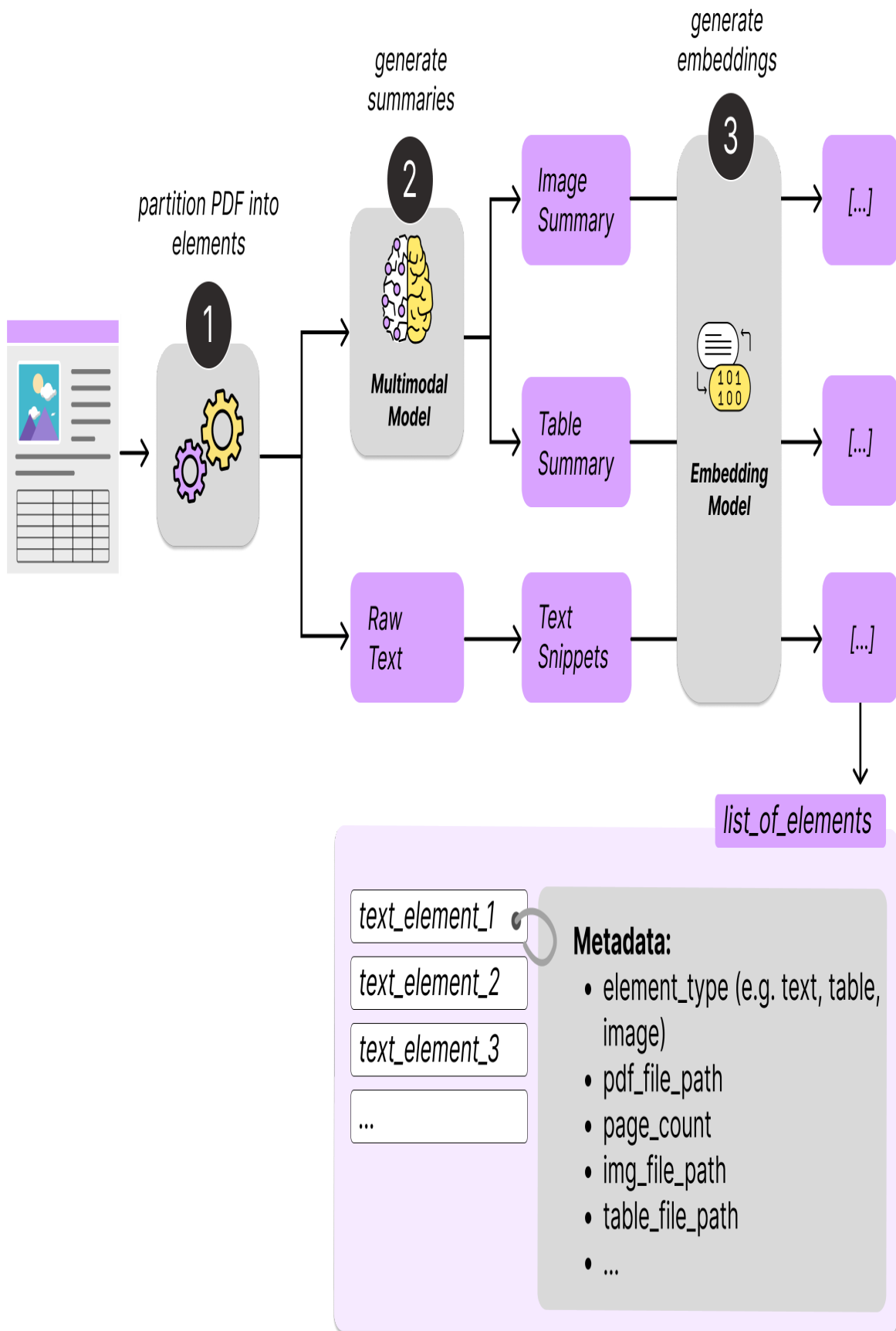


Figure 1-13. How to load and process multimodal PDFs.

Prerequisites

We need the **unstructured** library to break the PDF into text, images, and tables. After that, we use OpenAI's multimodal model **GPT-4o** to create text summaries for the images and tables.

Steps

In **Example 1-13**, we are using the **unstructured** library to partition the PDF into its elements. Each element has a particular category: narrative text, tiles, images, or tables.

Example 1-13. How to partition the PDF into text, images and tables

```
from unstructured.partition.pdf import partition_pdf
import os

# set the OCR agent to tesseract
os.environ["OCR_AGENT"] = "tesseract"

pdf_file_path = "../datasets/pdf_files/adult_data_article.pdf"
image_output_dir = "../datasets/extracted_content_from_pdfs/images"

# get elements using the function extract_pdf_elements
raw_pdf_elements = partition_pdf(
    filename=pdf_file_path,
    extract_images_in_pdf=True,
    extract_image_block_types=["Image", "Table"],
    extract_image_block_to_payload=False,
    extract_image_block_output_dir=image_output_dir,
)

# categorize elements by type
tables = []
texts = []
titles = []

# fill the just created lists with the elements
for element in raw_pdf_elements:
    if "unstructured.documents.elements.Table" in
str(type(element)):
        tables.append(str(element))
    elif "unstructured.documents.elements.NarrativeText" in
str(type(element)):
```

```
texts.append(str(element))
elif "unstructured.documents.elements.Title" in
str(type(element)):
    titles.append(str(element))
```

We get separate lists for each type of element. Here are some ideas for processing each type further:

- **Text and Titles:** Split them into smaller text chunks suitable for embedding models. This process involves techniques like recursive, semantic, and agentic chunking, which are detailed in [Chapter 2](#).
- **Images:** Extract text using OCR engines (see [Recipe 1.6](#)) or multimodal models (see [Recipe 1.7](#)). Alternatively, generate text summaries for the images using multimodal models (see [Recipe 1.8](#)).
- **Tables:** Either load the plain text from the table or, preferably, extract key insights using multimodal models (see [Recipe 1.9](#)).

Discussion

Many PDFs contain a mix of text, images, and tables. While humans can easily understand these elements, RAG systems often rely on text embedding models that excel at processing text but struggle with visual content. To bridge this gap, we can use multimodal models to analyze and summarize images and tables into text. This way, the text embeddings can capture the essential information from all elements.

See also

For more advanced loading pipelines for RAG systems, check out the blogs from major RAG frameworks and model providers like [LangChain](#), [Llamaindex](#), [OpenAI](#), or [Anthropic](#).

Here are two great blog posts to get you started:

- [How to parse PDF docs for RAG](#)

- [Multi-modal RAG Notebook Tutorial from LangChain](#) and [Semi-Structured RAG Notebook Tutorial from LangChain](#).

1.11 Loading Videos Using Speech-to-Text and Multimodal Models

Problem:

You want to load videos to your RAG system.

Solution:

We don't directly convert the video into vector data. Instead, we break it down into parts and turn each part into meaningful text chunks. These text chunks are then used to create embeddings with text embedding models.

In [Example 1-14](#), I'll show you how to load a video tutorial “[Learn Data Science](#)”. This is similar to many video meetings where a presenter explains something while showing PowerPoint slides.

[Figure 1-14](#) illustrates the steps in the loading pipeline:

1. **Partitioning:** Define timestamps for when the screen content changes significantly. Image processing libraries can help detect these changes. ()
2. **Save frames and create text summaries:** At each timestamp, save the video frame as an image and create a text summary (see [Recipe 1.8](#)).
3. **Transcribe audio using Speech-to-Text models:** Extract the audio segment between timestamps and save it as an mp3 file. Use the “Whisper” Speech-to-Text model to transcribe these audio files into text (see [Recipe 1.5](#)).

4. **Generate embeddings for frames and audio:** Use text embedding models to create embeddings for the text summaries of images and transcribed audio.
5. **Store text, images, and audio in the vector database:** Save the embeddings and metadata in the vector database.

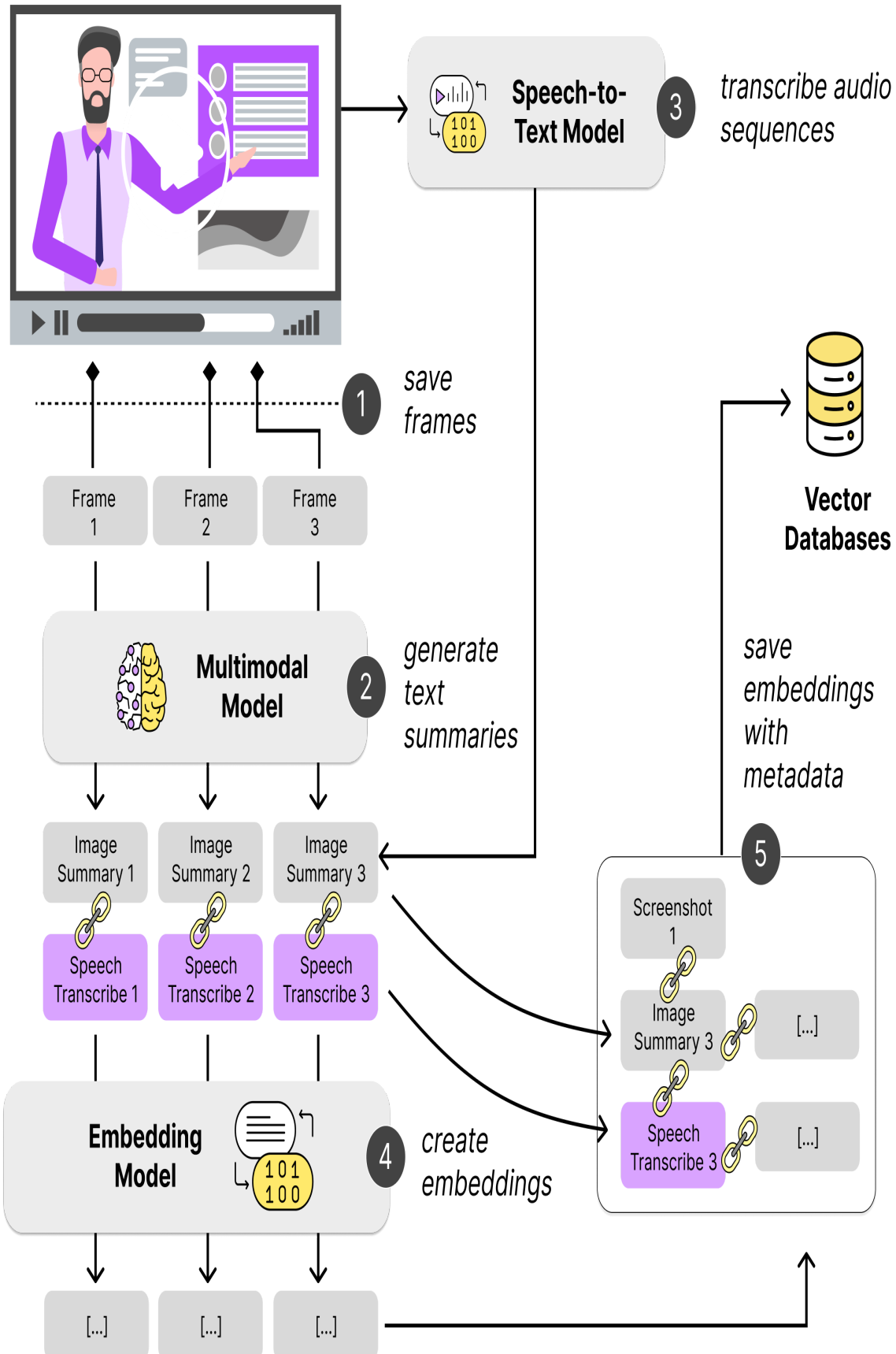


Figure 1-14. How to load videos into a RAG system by breaking them into their elements and processing them using speech-to-text and multimodal models.

Prerequisites

We will use the **moviepy** library to load the video, extract frames, and save audio clips. Moviepy is a Python library for video editing.

To process the video further, we will use two types of models:

- **Speech-to-Text Model:** OpenAI's Whisper model will convert the audio clips into text.
- **Multimodal Model:** OpenAI's GPT-4o model will create text summaries for the images.

Steps

To design an effective loading pipeline for the video, we need to consider its style.

Is it a single speaker talking, or a discussion between multiple people? Are there static slides on the screen or moving animations? How often do they change? If it's a simple slideshow, how frequently do the slides change?

The video we are loading in **Example 1-14** is a beginner Data Science tutorial. The speaker explains basic concepts using a slide presentation. The content on the screen changes infrequently, about once every 30 seconds.

Steps 1 and 2: Partition the video into sequences and save the frames as images

It makes sense to analyze the video (frames and audio track) to find the best timestamps for splitting the video and saving frames. For **Example 1-14**, we simplify the pipeline by defining a random list of timestamps. At each of these timestamps, we will use **moviepy** to save a frame from the video as an image.

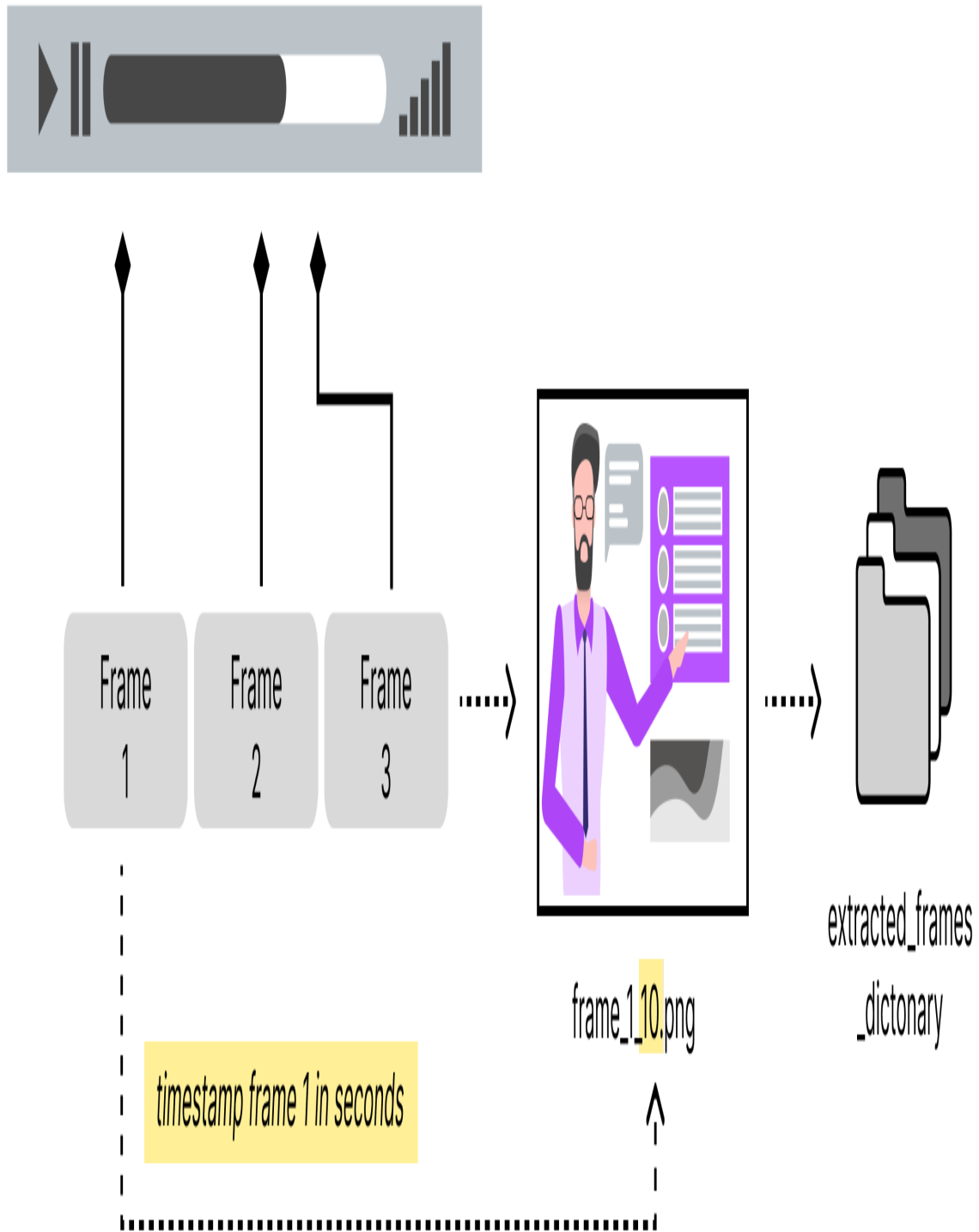


Figure 1-15. How to extract frames from the loaded video and save them as images

Example 1-14. Loading video files, extracting frames and saving them as

images

```
import os
from moviepy.editor import VideoFileClip
import pandas as pd

video_file_path = "../datasets/videos/learn-data-science-
tutorial.mp4"
image_output_folder = "../datasets/videos/video_extracted_images"

clip = VideoFileClip(video_file_path)

# create a list of timestamps from which we want to extract a frame
time_step = 10 # time in seconds
timestamps = list(range(0, int(clip.duration) - time_step,
time_step))

# for each timestamp extract a frame
for timestamp in timestamps:
    frame_image_path = os.path.join(image_output_folder,
f"frame_{timestamp}.png")
    clip.save_frame(frame_image_path, t=timestamp)
```

After running the code, you'll have a collection of images in the specified directory, each representing a frame from the video.

Now, let's decide how to process the saved images. Typically, we use multimodal models to describe the content. Here are some methods covered in earlier recipes:

- **Recipe 1.6** explains how to extract text from images using OCR.
- **Recipe 1.7** shows how to extract text from images using a multimodal model.
- **Recipe 1.8** demonstrates how to generate text summaries for images.

Step 3: Extract the audio between each of the timestamps and save it as an mp3 file

Next, we'll do the same for the audio track. **Example 1-15** iterates over the list of defined timestamps and saves the audio segments between two timestamps as mp3 files (**Figure 1-16**).

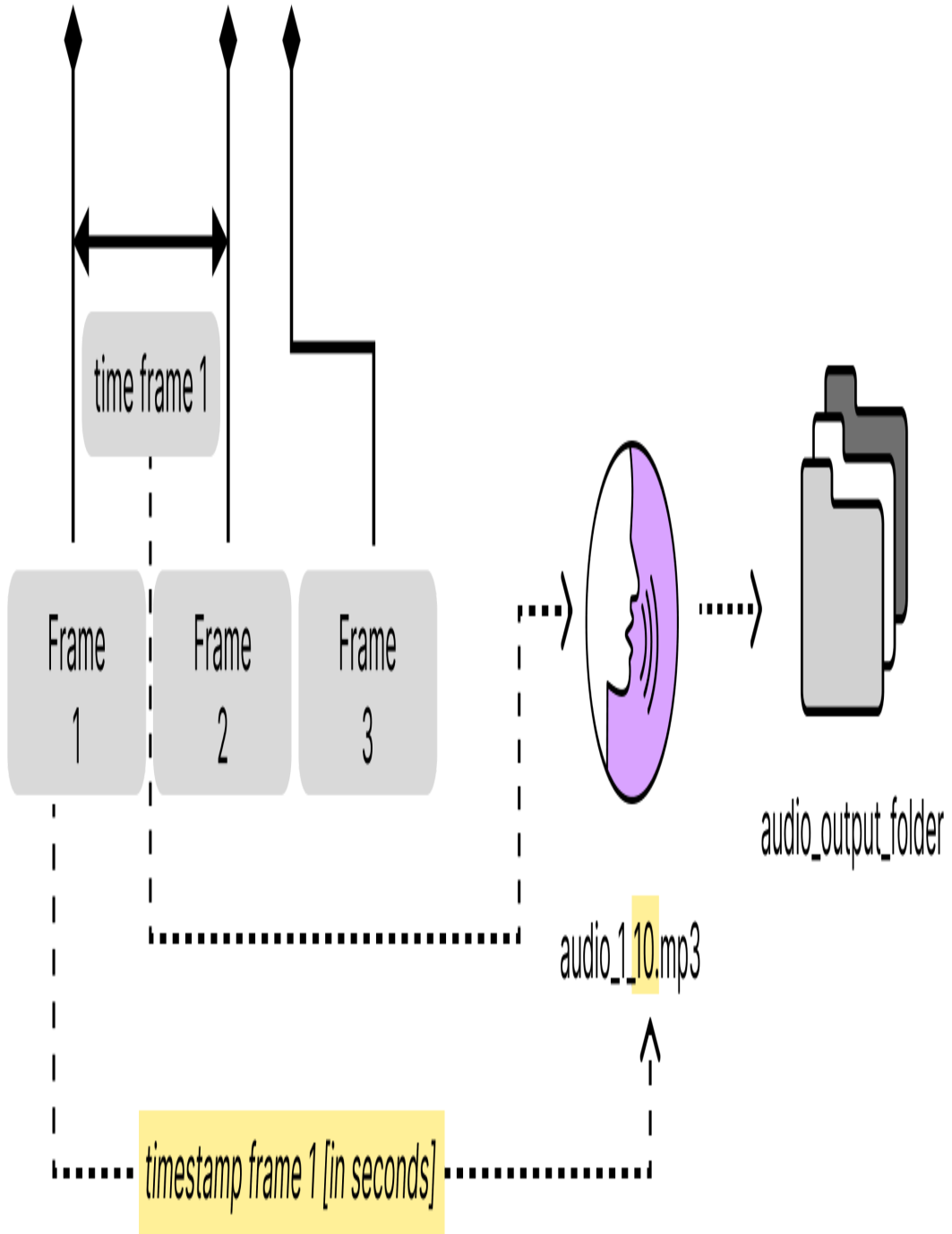


Figure 1-16. How to save audio sequences from the video to mp3 files.

Example 1-15. Extracting the audio between two timestamps and saving it as an mp3 file

```
# for each timestamp extract the audio sequence and save it to a
.mp3 file
audio_output_folder = "../datasets/videos/video_extracted_audio"

for timestamp in timestamps:
    audio_clip = clip.subclip(timestamp, timestamp +
time_step).audio
    output_audio_path = os.path.join(audio_output_folder,
f"audio_{timestamp}.mp3")
    audio_clip.write_audiofile(output_audio_path)
```

After splitting and saving the audio segments, we transcribe each audio file using the Whisper speech-to-text model. For more details, see [Recipe 1.5](#).

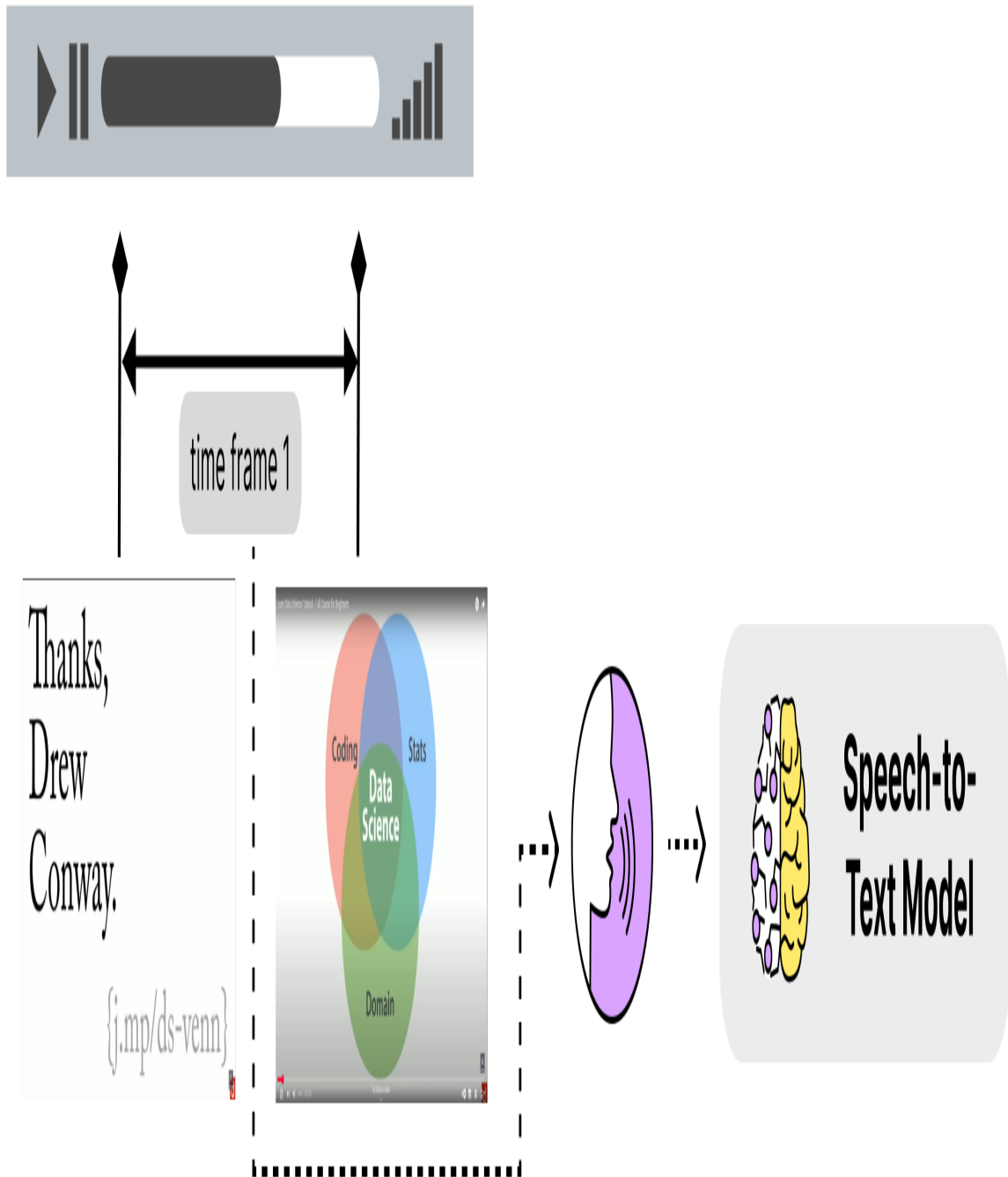


Figure 1-17. Transcribe the audio track between two saved frames from the “Data Science Tutorial” video.

The remaining tasks in Steps 4 and 5 are generating the embeddings and storing them along with the metadata in the vector database.

Discussion

I've learned half of what I know today from YouTube videos. I also spend several hours a day in video calls, many of which are recorded. But honestly, I rarely go back to watch old meeting recordings or YouTube videos unless I have a specific reason.

RAG systems can help extract useful insights from videos and turn them into a knowledge base that you can interact with. By breaking videos into sequences and frames, we can link directly to the original content and provide exact timestamps, making it much easier to revisit old videos.

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 2. Data Preparation

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at dmnkplzr@gmail.com.

In RAG systems, we break down longer texts into smaller chunks to make them manageable for text embedding models. These embeddings capture the meaning of the text as vectors in a multi-dimensional space (see [Link to Come] for how to create these embeddings). Later, we use these vectors to measure how similar two text chunks are by calculating the distance between them (see [Link to Come] for details).

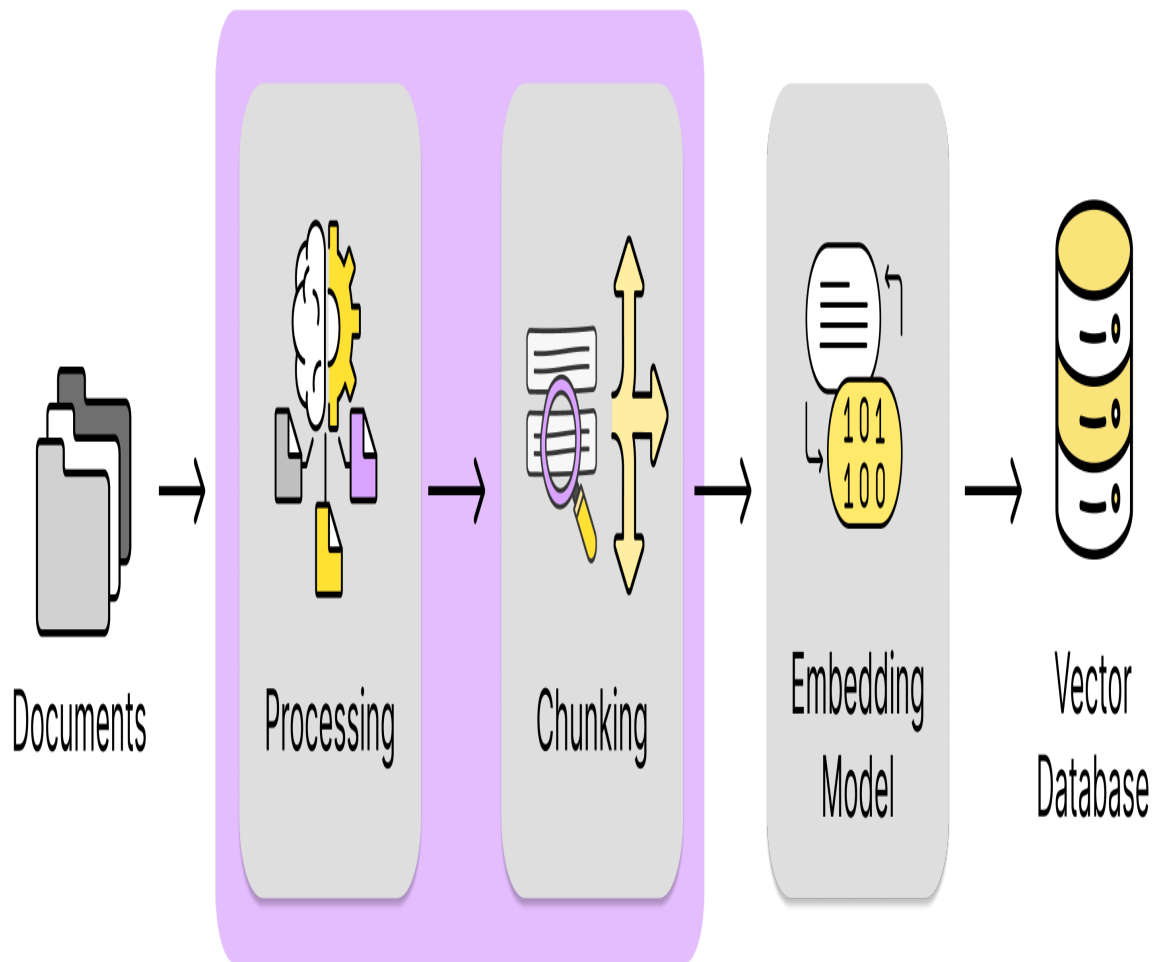
To ensure our RAG system finds the relevant information effectively, we need to create text chunks that clearly capture individual pieces of information. A strong data processing pipeline helps by cleaning up the raw text and splitting it at the right points to create meaningful chunks.

Figure 2-1 shows some techniques for these steps.

Here are the steps in the pipeline:

- **Clean the raw text:** Remove unclear abbreviations to make each text chunk self-explanatory. See **Recipe 2.2** for more details.

- **Collect and store metadata:** Gather metadata, like the author's name or the document's creation date, to help filter out irrelevant information during retrieval. See [Recipe 2.1](#) for how to collect metadata fields from the document and the text itself.
- **Split the text into smaller chunks:** The simplest method is using a character splitter (see [Recipe 2.4](#)), which divides the text into fixed-size chunks. However, it's better to use a method that considers the document's structure, like recursive chunking (see [Recipe 2.5](#)), which splits the text at paragraph or heading breaks. For unstructured documents, like audio transcriptions or chat histories, try semantical chunking (see [Recipe 2.7](#)) or agentic chunking (see [Recipe 2.8](#)). These methods analyze the content to decide where to split the text, though they are more complex and costly than recursive chunking.



- Metadata
- Technical Terms
- Abbreviations
- Summaries
- Hypothetical Questions
- Recursive Chunking
- Document Specific Chunking
- Semantic Chunking
- Agentic Chunking

Figure 2-1. The simplified RAG indexing pipeline including potential data processing techniques

2.1 Adding Metadata to Enable Metadata Filtering

Problem

You want to store useful metadata with each text chunk in your vector store. This helps you apply filters when searching for relevant content, making the retrieval step in your RAG system faster and more accurate.

Solution

We can extract and generate metadata in two ways: from the document's stored metadata or by using LLMs to scan the document text. [Figure 2-2](#) shows the three steps to build a detailed metadata dictionary:

1. **Extract metadata:** Extract the existing metadata from the document. See [“Step 1: Extract the metadata stored in the document”](#).
2. **Add metadata:** Add extra metadata like file location, size, or page count. Refer to [“Step 2: Add extra metadata like extraction time, file size, and more”](#).
3. **Generate new metadata:** Use LLMs to find metadata in the document's text, such as the author's name and contact details. For contracts, look for specific clauses. See [“Step 3: Generate new metadata by analyzing the document text”](#) for details.

Start by checking the existing metadata in the document and then fill in the gaps with information from the text. PDFs store metadata in XML format, which often includes details like the author, creation date, and last modification date. However, they usually lack a description of the document type. For example, in a RAG system for HR analyzing resumes,

it would be helpful to label each resume by job type, such as software developer, graphic designer, or marketing specialist. “**Step 3: Generate new metadata by analyzing the document text**” shows how to use LLMs to fill this gap by analyzing the file content. This way, if a user asks, “*What skills are commonly mentioned in software developer resumes?*“, the system can filter out non-developer resumes.

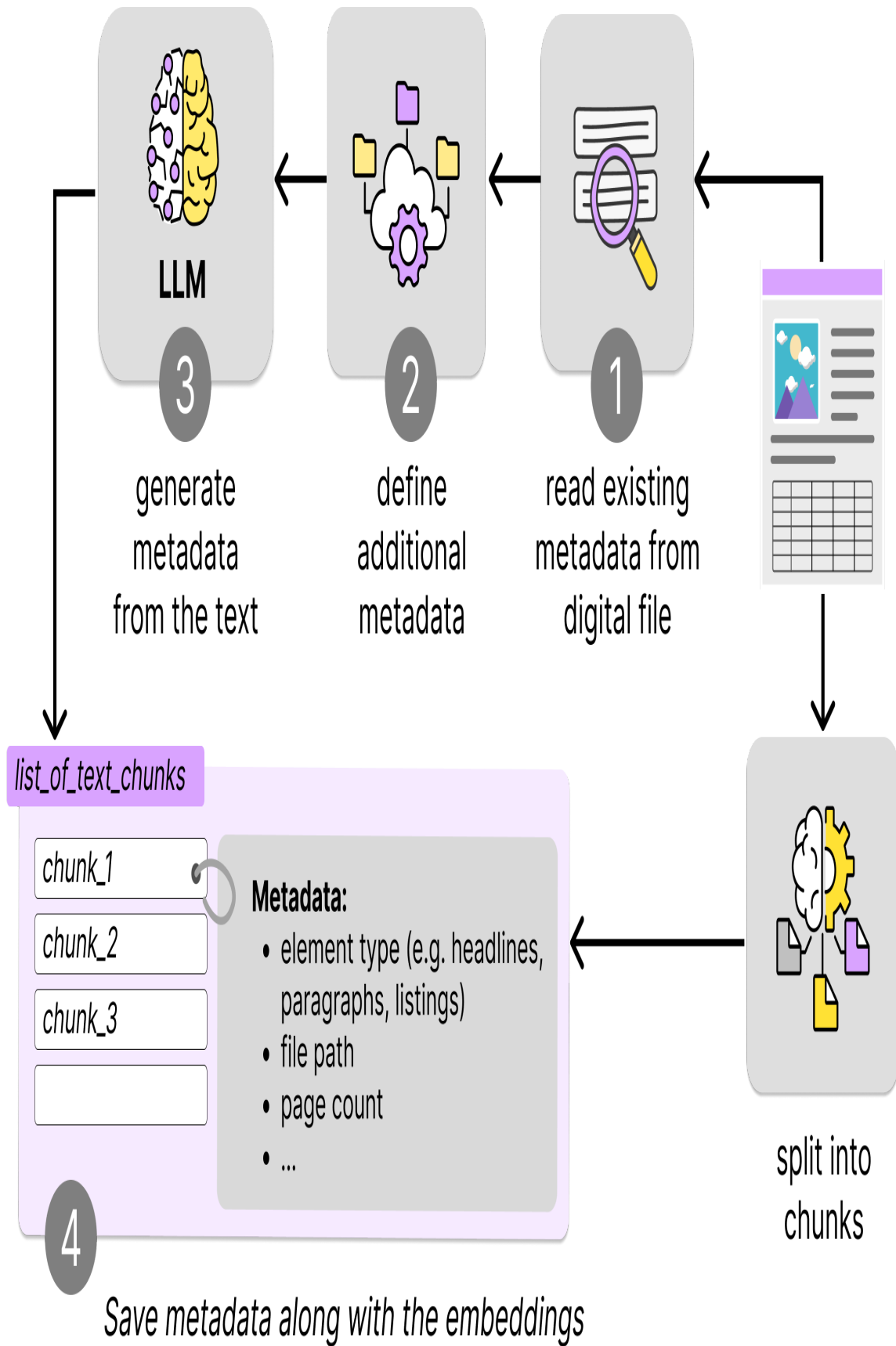


Figure 2-2. How to extract and generate metadata fields

Prerequisites

This recipe uses the PyPDF2 library to extract metadata from PDFs and the OpenAI API to extract metadata from the document text.

Steps

Following the process shown in [Figure 2-2](#), we have three steps to build a comprehensive metadata dictionary:

Step 1: Extract the metadata stored in the document

In [Example 2-1](#), we use the PyPDF2 library to load the PDF and extract the metadata stored within it. Common metadata fields include the author, creation date, last modification date, creator (software used to create the document), producer (software used to convert the document to PDF), title, and subject. Not all documents will have all these fields, but this is a good starting point to check what's available.

Example 2-1. How to extract metadata from a PDF

```
file_path =
    "../..../datasets/pdf_files/attention_is_all_you_need_paper.pdf"

with open(file_path, "rb") as file:
    reader = PyPDF2.PdfReader(file)
    metadata = reader.metadata

    text = ""
    for page in reader.pages:
        text += page.extract_text()
```

Step 2: Add extra metadata like extraction time, file size, and more

Besides the standard metadata fields, you can include any other useful information. [Example 2-2](#) demonstrates how to add values for the PDF's page count, file size, file name, file path, and the length of the extracted text.

Example 2-2. How to add customized metadata fields to the metadata

dictionary

```
metadata = dict(metadata)
metadata["page_count"] = len(reader.pages)
metadata["file_size"] = os.path.getsize(file_path)
metadata["file_name"] = os.path.basename(file_path)
metadata["file_path"] = file_path
metadata["text_length"] = len(text)
```

Step 3: Generate new metadata by analyzing the document text

When you run the code in **Example 2-2**, you'll notice that the author's name fields are empty. However, these names are usually present somewhere in the document, especially in scientific papers. To extract this information, we need to analyze the document text directly.

Example 2-3 demonstrates how to use LLMs to find and fill in this metadata. We load the entire text and let the LLM search for the relevant information. To ensure the LLM provides only the required metadata and avoids any irrelevant details, we use OpenAI's structured output functionality. This involves defining a Pydantic model that specifies the expected data fields and their formats. Pydantic is a popular tool for data validation, helping us clearly outline the expected values and formats.

Additionally, we need to instruct the model on what to do. These instructions, known as the system message, are placed at the beginning of the prompt.

Figure 2-3 illustrates the steps to generate a structured output using a Pydantic model.

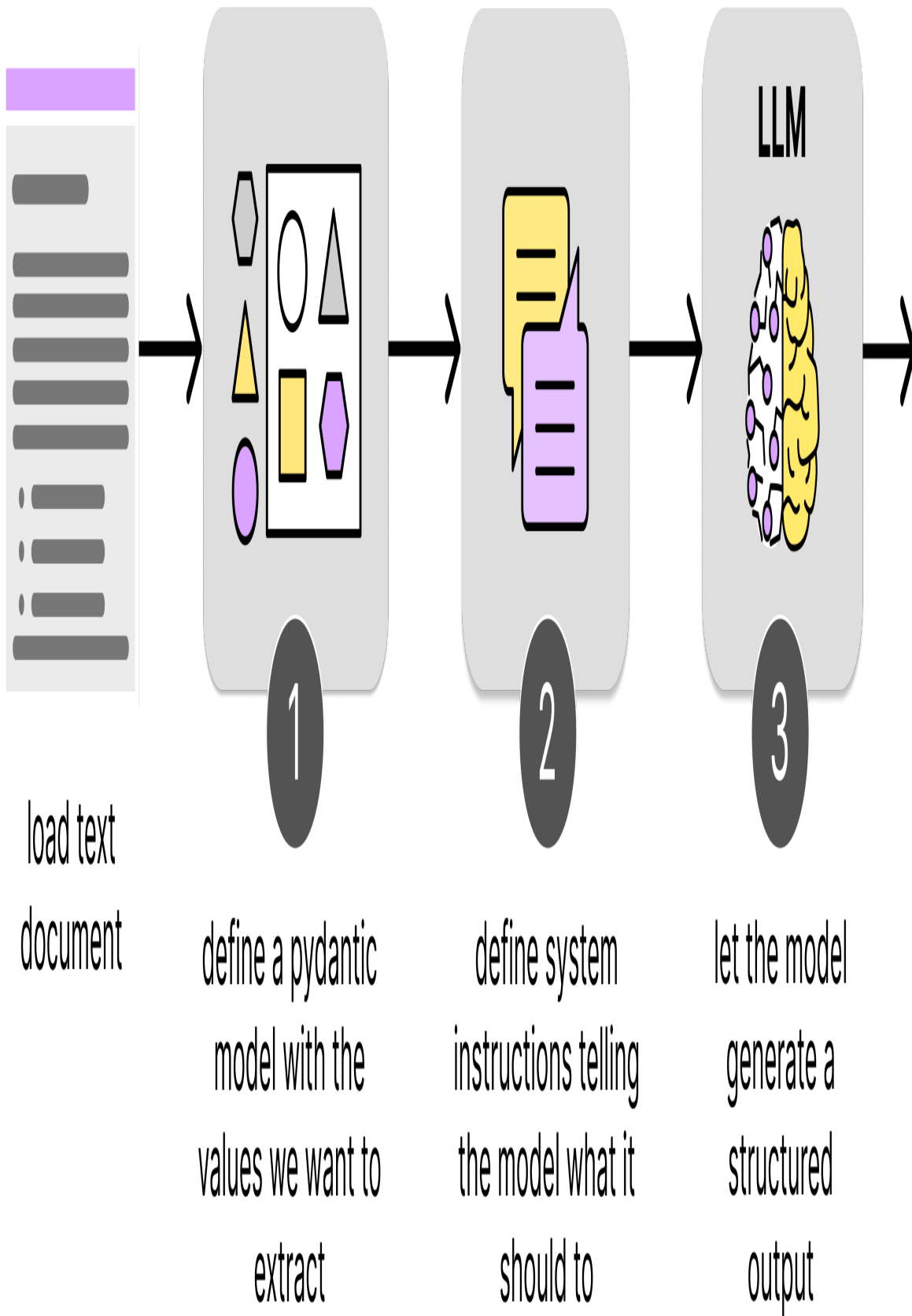


Figure 2-3. Generate metadata from the document text by using LLMs

Example 2-3 processes a scientific paper where the author names are not in the PDF's metadata. We use a Pydantic model called `AuthorContact` to specify the data fields we want, like author names, email addresses, and company names. Since there can be multiple authors, we use another model called `Contacts` to hold a list of `AuthorContact` objects. The LLM scans the text and extracts the specified fields.

Example 2-3. Generate metadata from the document text using LLMs

```
from pydantic import BaseModel
from openai import OpenAI

client = OpenAI()

class AuthorContact(BaseModel):
    name: str
    company: str
    email: list[str]

class Contacts(BaseModel):
    entries: list[AuthorContact]

system_message = """Extract the contact information of all
authors."""

completion = client.beta.chat.completions.parse(
    model="gpt-4o-2024-08-06",
    messages=[
        {
            "role": "system",
            "content": system_message,
        },
        {
            "role": "user",
            "content": text,
        },
    ],
    response_format=Contacts,
)

author_contacts = completion.choices[0].message.parsed

metadata["author_contacts"] = author_contacts
```

The code snippet returns a list of dictionaries. Each dictionary contains details about an author, such as their name, company, and email address. This list is then added to the metadata dictionary.

Discussion

Metadata helps us narrow down the search space in our vector database before performing a vector search. This can speed up the search and filter out irrelevant information. Not every text chunk that shows high semantic similarity to the user's question is actually relevant.

Figure 2-4 shows an example of a common challenge during the retrieval step. Both statements in the image seem relevant, but:

- If you are a doctor, you are interested in the benefits of antioxidants for the body.
- If you are an engineer, you are interested in the positive impact of antioxidants on the characteristics of raw materials.

You will rarely have a user ask for other scenarios. Thus, information that seems correct and relevant at first glance can be irrelevant or even wrong when we know the full context behind the user's question.

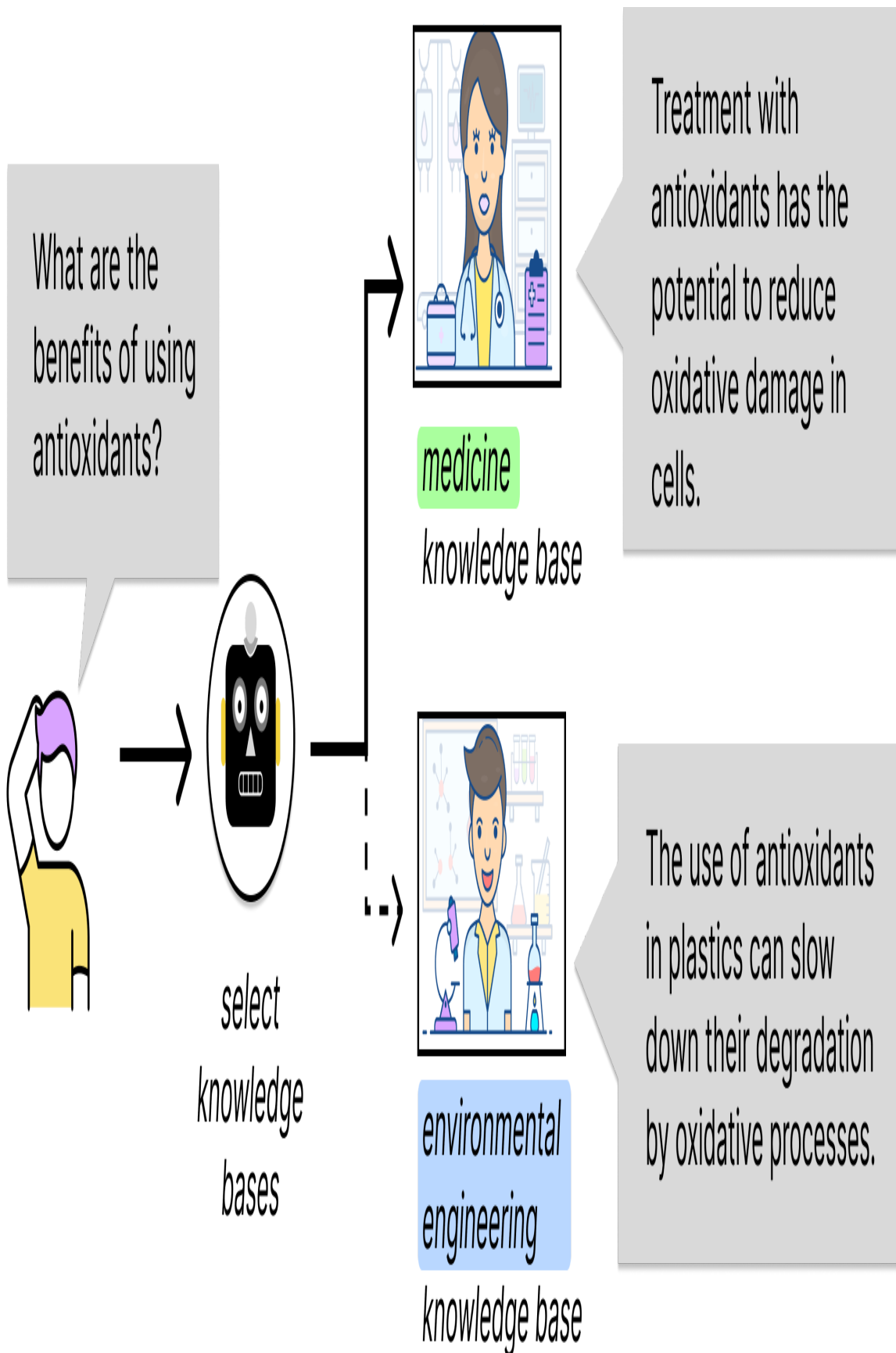


Figure 2-4. How two semantically similar information sets can be contradictory

The example in **Figure 2-4** is extreme, but the issue is common when building RAG apps. We often deal with documents that seem related at first glance, but when we look closer, there is one fact that makes them completely irrelevant.

Other filter criteria could be the period the documents were published, the author's name, whether certain celebrities were mentioned in the text, or whether contracts contain specific clauses. What filter criteria could be helpful really depends on your specific use case.

2.2 Enhancing Data Quality by Replacing Abbreviations and Technical Terms

Problem

You want to make text chunks more self-explanatory by replacing domain-specific abbreviations. This ensures that our embedding model and LLMs clearly understand the meaning behind the text snippets.

Solution

Text chunks are segments of a larger text that can be difficult to understand when viewed in isolation.

Figure 2-5 illustrates some challenges that the RAG system may face without proper context.

- **Challenge 1:** Your document contains many domain-specific abbreviations, making it hard to understand when only parts of the text are visible. For example, in the first column of **Figure 2-5**, “purchase order” is abbreviated as “PO” in a supply chain document. However, depending on the context, this abbreviation could also mean “Product Owner” or “Post Office”. Similarly, in the sports example in the middle of the figure, “MJ” stands for

“Michael Jordan”. If you are not familiar with sports or basketball, you might not recognize who MJ is. Learn how to address this in [Recipe 2.2](#).

- **Challenge 2:** You are dealing with documents that contain many images, such as tutorials with screenshots. This can be challenging because most basic RAG systems only process text. For instance, a tutorial might refer to a “settings” button without showing its location or appearance. Learn how to address this issue using multimodal models in [“Challenge 2: Handling documents with many images”](#).

Purchase Order Status

"The PO was created and sent to the supplier 'Bearings and More AG'"



Better

"The Purchase Order (PO) was created and sent to the supplier 'Bearings and More AG'"

Sports News

"MJ is the best basketball player of all times"



Better

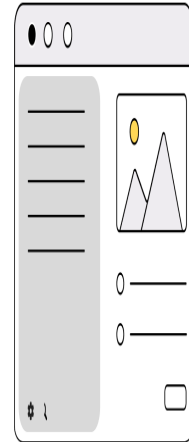
"Michael Jeffrey Jordan, often just called MJ, is the best basketball player of all times"

Tool Tutorial

Switch to light mode

1. Click on settings"

2. ...



Switch to light mode Better

When you open the main page of the website, you will see a cogwheel icon in the bottom corner of the screen. Click on it to open the settings page ...

Figure 2-5. The way that documents are structured influences the performance of our RAG system

These challenges are so nuanced, that there is not one solution that fits all. We need to tackle them step by step.

Challenge 1: Handling documents with many domain-specific abbreviations

Example 2-4 demonstrates how to use a predefined abbreviation dictionary to replace abbreviations in text chunks.

For example, consider a blog post about transformer models filled with abbreviations like *LSTM* (Long Short-Term Memory) and *FFN* (Feed-Forward Network). Even experienced data scientists might not know all these terms. The process in **Example 2-4** replaces abbreviations with their full terms and adds the abbreviation in brackets, e.g., *LSTM* becomes *Long Short-Term Memory (LSTM)*. This ensures that our RAG system can understand user queries regardless of whether they use abbreviations or full terms: *What is LSTM?* vs. *What is Long Short-Term Memory?*.

Example 2-4. Enhancing text chunks by replacing abbreviations

```
import re

abbreviations_dict = {
    "NLP": "Natural Language Processing",
    "RNN": "Recurrent Neural Network",
    "LSTM": "Long Short-Term Memory",
    "GRU": "Gated Recurrent Unit",
    "TF": "Transformer",
    "MHA": "Multi-Head Attention",
    "FFN": "Feed-Forward Network",
}

# Load the sample text file
file_path = "../..../datasets/text_files/blog_post_transformers.txt"
with open(file_path, "r") as file:
    text = file.read()

# Replace abbreviations in the text
for abbr, full_form in abbreviations_dict.items():
    text = re.sub(rf"\b{abbr}\b", f"{full_form} ({abbr})", text)
```

Challenge 2: Handling documents with many images

For more complex tasks, we need to use LLMs and multimodal models.

Example 2-4 loads the text from a **PowerPoint slide about financial figures**.

The content is filled with numbers and finance-specific abbreviations.

Without a finance background and the accompanying graphs, it's challenging to fully grasp the slide's information.

We use OpenAI's multimodal model GPT-4o to interpret the content, add context, and replace abbreviations and technical terms to make it easier to understand.

Example 2-5. Enhancing text chunks using LLMs

```
import os
from openai import OpenAI

file_path = "../../datasets/text_files/EMEA_drives_revenue.txt"
with open(file_path, "r") as file:
    text = file.read()

prompt = f"""
    The text below contains a financial report including a lot of
    abbreviations and
    technical terms from the finance domain. Please replace the
    abbreviations with
    their full forms and provide a brief explanation of the
    technical terms, so the whole
    text get's easier to read and understandable for everyone.

    Make sure it's easy enough, that a 10 years old school kid
    could understand it.

    Often used abbreviations are:
    - EMEA: Europe, Middle East, and Africa
    - BD: Business Development
    - YoY: Year-over-Year
    - APAC: Asia-Pacific

    Text:
    {text}
    """.strip()

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
chat_completion = client.chat.completions.create(
    messages=[
```



```
        {  
            "role": "user",  
            "content": prompt,  
        }  
    ],  
    model="gpt-4o",  
)
```

Discussion

In an ideal world, text chunks would be easy to understand on their own. This would allow anyone to grasp the content just by reading the snippet. However, in reality, we often deal with complex texts filled with domain-specific abbreviations and references to other parts of the document.

Additionally, most documents require some background knowledge to fully understand the information. For example, reading an advanced Python book assumes you already know basic Python concepts. If you encounter something you don't understand, you might refer to a beginner book to fill in the gaps. Gradually, you build up the knowledge needed to fully comprehend the document.

In a basic RAG system, the LLM only sees the information retrieved from the vector search. This recipe showed techniques to replace technical terms, making text chunks easier to understand. Here are some additional tips:

1. **Adding Contextual Sentences:** Instead of “It improved efficiency by 40%.”, use “The new indexing algorithm improved search efficiency by 40%.”
2. **Avoiding Pronouns and Implicit References:** Instead of “It was implemented last year.”, use “The customer feedback system was implemented last year.”
3. **Preserving Full Names of Entities:** Instead of “Google launched it in 2019.”, write “Google launched the BERT language model in 2019.”

4. **Avoiding Ambiguous Words:** Instead of “The system works better now.”, write “The system’s response time improved from 500ms to 200ms after optimization.”

See also

In the discussion section of this recipe, I have described several issues that make text chunks difficult to comprehend.

Instead of addressing each challenge separately, we can use LLMs to handle most of these issues by scanning the entire document and forming text chunks that are complete, meaningful, and clear. Agentic chunking (see [Recipe 2.8](#)) does exactly that by identifying similar information across documents and building propositions that contain all relevant details. These propositions serve as high-quality text chunks. The only downside is that it can become quite costly to scan, analyze, and process every single document using LLMs. You need to decide if the increased quality of the text chunks is worth the additional cost.

2.3 Improving Search Accuracy by Embedding Hypothetical Questions

Problem

You want to address the mismatch between user questions and document content in the embedding space by creating hypothetical questions.

Solution

We have previously used LLMs to pre-process raw text for our RAG system. For example, in [Recipe 1.8](#), we used a multimodal model to create text summaries from images. In [Recipe 1.9](#), we used an LLM to extract key insights from tables.

In this recipe, we reverse the process. Instead of summarizing content, we show the LLM a final text chunk and ask it to generate potential user questions that can be answered using that chunk. [Figure 2-6](#) illustrates this process step by step.

1. **Load data:** Load the data and split it into chunks. Recursive chunking is a good starting point; see [Recipe 2.5](#).
2. **Generate hypothetical questions:** Iterate over the chunks and generate hypothetical questions for each one. In [Example 2-6](#), we create five questions per chunk.
3. **Generate embeddings:** Generate embeddings for each hypothetical question (see [\[Link to Come\]](#)).
4. **Push to database:** Store the hypothetical questions and their embeddings in the database. Each question links to the original text chunk in the metadata. This is crucial because the original text chunk, not the hypothetical questions, will be provided to the LLM later. The hypothetical questions are used solely to enhance the retrieval step.

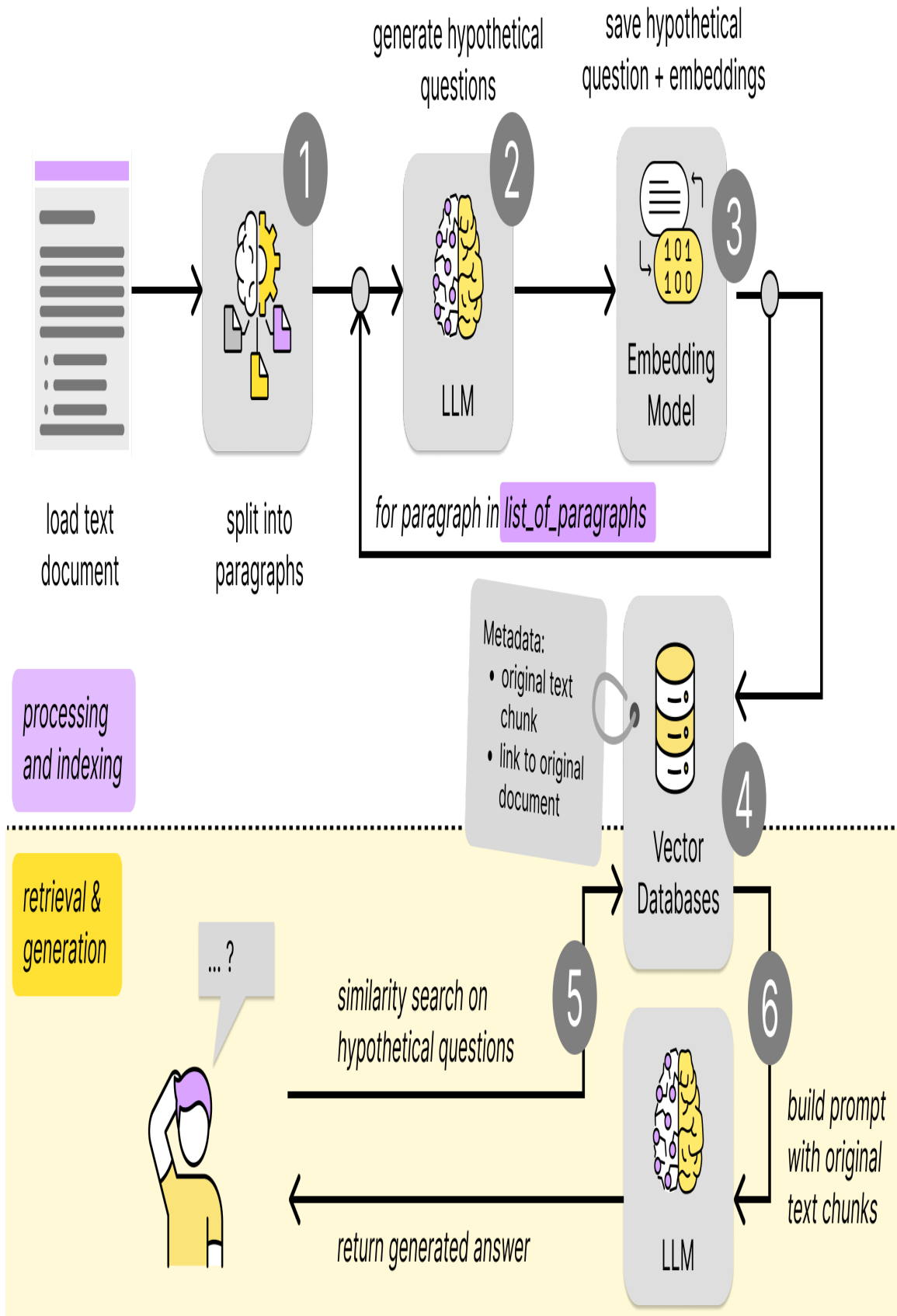


Figure 2-6. How to generate hypothetical questions using LLMs

The purpose of this process is to enhance the retrieval step. When a user asks a question, follow these steps:

5. **Generate embeddings:** Generate embeddings for the user's question to compare them with the hypothetical questions.
6. **Retrieval:** Retrieve a list of hypothetical questions that are semantically similar to the user's question. Use the original text chunks associated with these hypothetical questions to build an appropriate prompt.

Prerequisites

To generate effective hypothetical questions, we need a capable LLM. If budget allows, using a powerful LLM is recommended to ensure high-quality questions that enhance retrieval accuracy. In [Example 2-6](#), we will use OpenAI's GPT-4o.

Steps

In [Example 2-6](#), we load a [sample chat history](#) between two students discussing the usefulness of AI in manufacturing. As shown in [Figure 2-6](#), we go through the text chunks and ask the LLM to generate hypothetical questions for each chunk. To help the LLM generate relevant questions, we can provide additional context, such as information about the document, its type, or its domain.

To ensure we receive a list of hypothetical questions formatted as a Python list of strings, we use the structured output functionality. In [Example 2-6](#), we define the expected structure using the Python class `HypotheticalQuestions`.

Example 2-6. How to generate hypothetical questions using a LLM

```
import os
from openai import OpenAI
from pydantic import BaseModel
import textwrap
```

```

file_path = "../../datasets/text_files/AI_in_factories_chat.txt"

with open(file_path, "r") as file:
    text = file.read()

client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

prompt = textwrap.dedent(
    f"""
    Below you can find a chat history between two students.

    Please generate 5 hypothetical questions that could be
    answered using the information from the discussion.
    The questions should focus on key details, definitions, and
    information present in the text.

    Chat History:
    {text}
    """
)

class HypotheticalQuestions(BaseModel):
    questions: list[str]

# generate hypothetical questions using the GPT-4 model
completion = client.beta.chat.completions.parse(
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
    model="gpt-4o",
    response_format=HypotheticalQuestions,
)

hypothetical_questions =
completion.choices[0].message.parsed.questions

```

Now, we generate embeddings for each hypothetical question and store them in our vector database. It's important to also save the original text chunk and useful metadata, such as the link to the original document and the page number where the text chunk is found.

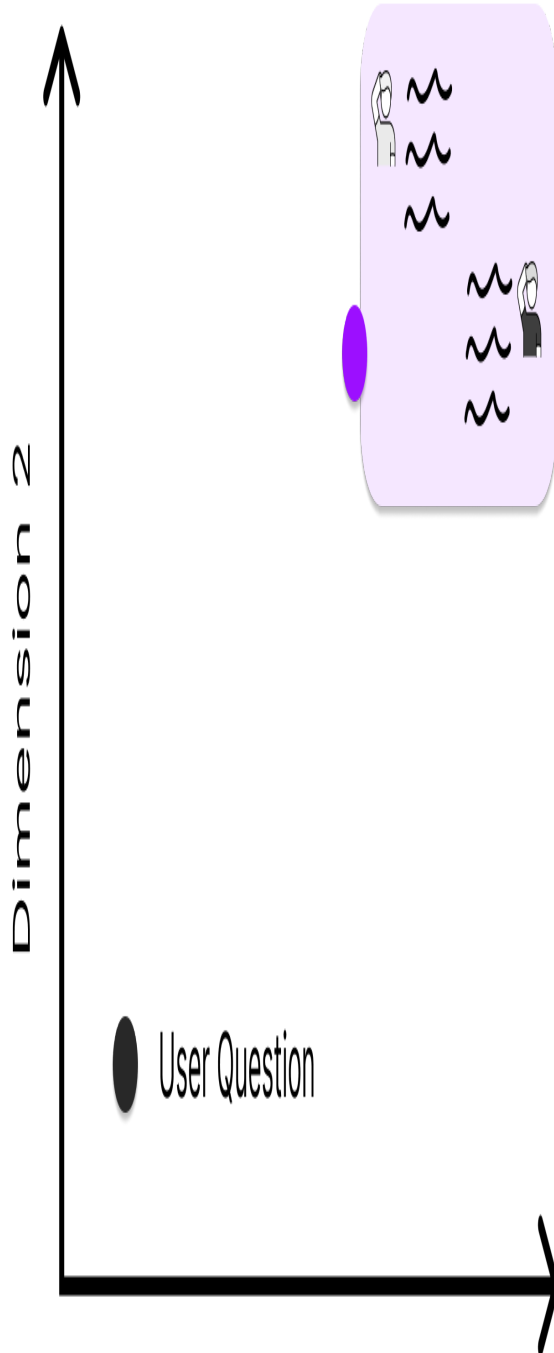
Discussion

To improve the accuracy of our RAG system, enhancing the retrieval step is crucial.

In a basic RAG setup, we directly compare user questions with text chunks. The challenge is that these text chunks can vary widely, including code snippets, text paragraphs, or tables. Because of the inherent differences between a question and, for example, a table or chat history, the vector embeddings may not align perfectly, even if the underlying information matches well. This issue is often referred to as a *semantic alignment problem* (see the left side of [Figure 2-7](#)).

original document

e.g. chat history, source code, etc.



Original document
e.g. chat history, source
code, etc.

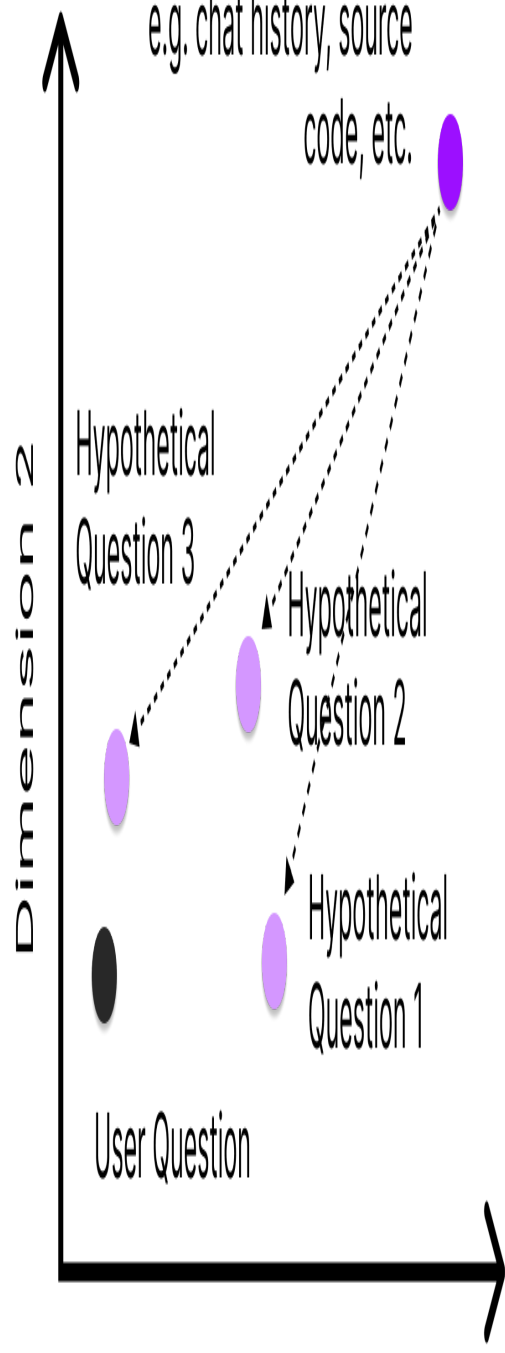


Figure 2-7. Hypothetical questions are closer to the location of potential user questions in the embedding space

By creating hypothetical questions, we bridge the gap between user queries and document content. These questions highlight the specific information available in the documents, making it easier to retrieve the relevant ones. During the generation step in our RAG pipeline, we use the original text chunks linked to these hypothetical questions.

2.4 Splitting Documents Using Character Splitting

Problem

You need to break down lengthy documents into smaller sections to create embeddings and analyze similarities.

Solution

The Character Text Splitter is the most straightforward method for dividing text. It uses a fixed-size window to create chunks of equal length (see [Figure 2-8](#)).

While this method is quite basic and often not the best choice, it can be useful in specific scenarios. For instance, when dealing with raw log files or real-time data streams that lack clear sentence or paragraph breaks, character splitting ensures consistent chunk sizes. This makes processing simpler and helps keep the text within the token limits of foundation models. For most other cases, consider using recursive chunking (see [Recipe 2.5](#)) or document-aware chunking (see [Recipe 2.6](#)).

chunk 1

2023 text-generating language model

chunk 2

Generative Pre-trained Transformer 4 (GPT-4) is a multimodal large

language model created by OpenAI, and the fourth in its numbered

"GPT-n" series of GPT foundation models. It was released

chunk 3

Figure 2-8. Splitting text using a fixed character or token size

Prerequisites

We are using [LangChain's CharacterTextSplitter](#). ===== Steps

This snippet uses the text splitter module from LangChain. The [Example 2-7](#) counts characters, sets a chunk size of 100 with zero overlap, and uses a space (' ') as the separator. This ensures that words aren't split in the middle while keeping chunks close to the 100-character limit.

The goal is to create text chunks that are long enough to give the LLM sufficient context but still small enough to generate specific embeddings that capture individual pieces of information. Here, I've used 100 characters to demonstrate the concept, but in real RAG use cases, chunk sizes between 1000-2000 tokens or around 4000-8000 characters are typically chosen (since one token is roughly 4 characters in English). It's important to stay within the token limits of the embedding models. For example, the text-embedding-3-small model has a context length of 8191 tokens, so your chunks can be up to 8191 tokens. However, keep in mind that larger text chunks lead to larger prompts sent to the LLM, which can slow down response time and increase running costs.

Example 2-7. How to split text using a fixed text chunk size

```
from langchain.text_splitter import CharacterTextSplitter

file_path = "../../datasets/text_files/blog_post_transformers.txt"

# Load example document
with open(file_path, "r") as file:
    text = file.read()

text_splitter = CharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=0,
    separator="",
    length_function=len,
)

text_splitter.create_documents([text])
```

Discussion

The character splitter is a simple way to split the text into chunks of a specific size. However, this method also has many constraints. It doesn't take the structure of the document into account at all. Instead, it splits paragraphs and sentences somewhere in the middle. While doing that, we are ripping not only the sentences and paragraphs apart but also the text's whole meaning. Every effort the author invested in properly structuring the document is lost. There are better ways; you can find some in “[See Also](#)”.

See Also

Visualizing the splitting output helps one understand the different chunking methods. A simple web app for the LangChain splitters does just that: chunkviz.up.railway.app.

You will likely find that the character splitter is not the best choice for your needs. While it may seem simple or efficient, its limitations often outweigh its benefits. Better alternatives include:

- For classic text documents with headings, chapters, and paragraphs, use recursive chunking, which respects the document's structure (see [Recipe 2.5](#)).
- For documents created with markup languages like Markdown, HTML, or LaTeX, use document-aware chunking, which understands the syntax of these languages (see [Recipe 2.6](#)).
- For documents without a clear structure, such as audio transcriptions or chat histories, use semantical chunking, which analyzes the content to decide where to split the text (see [Recipe 2.7](#)).
- For complex documents that frequently reference other parts, use agentic chunking. This method generates standalone statements called propositions and merges similar information into one text chunk. It is very powerful but also costly (see [Recipe 2.8](#)).

2.5 Splitting Documents Using Recursive Text Splitters

Problem

You need to load and split traditional text documents that have a clear structure with headings, chapters, and paragraphs.

Solution

The recursive text splitter identifies characters that indicate the document's structure, such as new lines, spaces, and punctuation marks. By default, it uses two new lines (“\n\n”), one new line (“\n”), spaces (“ ”), and punctuation marks (“.”). **Figure 2-9** illustrates how a simple text snippet is divided into chunks using recursive chunking.

chunk 1

2023 text-generating language model

→ Split at [n]

chunk 2

Generative Pre-trained Transformer 4 (GPT-4) is a multimodal large

language model created by OpenAI, and the fourth in its numbered

"GPT-n" series of GPT foundation models

It was released

→ Split at [.] and
chunk size

chunk 3

Figure 2-9. Splitting text using recursive chunking

Prerequisites

We are using the `RecursiveCharacterTextSplitter` from LangChain.

Steps

Example 2-8 processes text from a sample news page covering politics, sports, and healthcare. To ensure each chunk is of a similar size, we set a minimum chunk size. The splitter counts characters or tokens, and once it reaches the defined size, it looks for the next separator to split the text. In this example, we use a small chunk size of 200 characters. For most RAG applications, a chunk size of 1000-2000 tokens (approximately 4000-8000 characters) is more suitable. The maximum chunk size depends on the token limit of the text embedding model and LLM you are using.

Example 2-8. Recursive character text splitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

file_path = "../../datasets/pdf_files/daily_insights.pdf"

with open(file_path, "rb") as file:
    reader = PyPDF2.PdfReader(file)

    text = ""
    for page in reader.pages:
        text += page.extract_text()

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0,
    length_function=len,
    is_separator_regex=False,
)

chunks = text_splitter.split_text(text)
```

The text snippet generates text chunks that are roughly 200 characters each. However, the actual length can vary. If the text has many long sentences and the recursive chunker splits after each sentence, some chunks might exceed 200 characters.

NOTE

The default list of separators works well for most texts and languages. However, for languages like Chinese, Japanese, and Thai, which lack word boundaries, these separators might split words incorrectly. To avoid this, you can customize the list of separators. Best practices can be found in the [LangChain documentation](#).

Discussion

Ideally, each text chunk should cover a single idea. When the author finishes one thought and starts another, we should end one chunk and begin a new one. Fortunately, humans often structure texts to reflect their meaning, grouping related information into paragraphs and chapters with clear headings.

Figure 2-10 illustrates the process of loading and splitting a sample news page, where each section covers the latest news on a specific topic. Randomly creating text chunks can mix information from different sections. This makes it harder for embedding models to capture the semantic meaning accurately. Even if the retrieval step works well, we might load irrelevant information into our prompt, increasing costs, slowing response time, and making it harder for the LLM to generate a good answer. For example, randomly splitting the news page from **Example 2-8** could mix information from politics and sports, adding noise to our prompt.

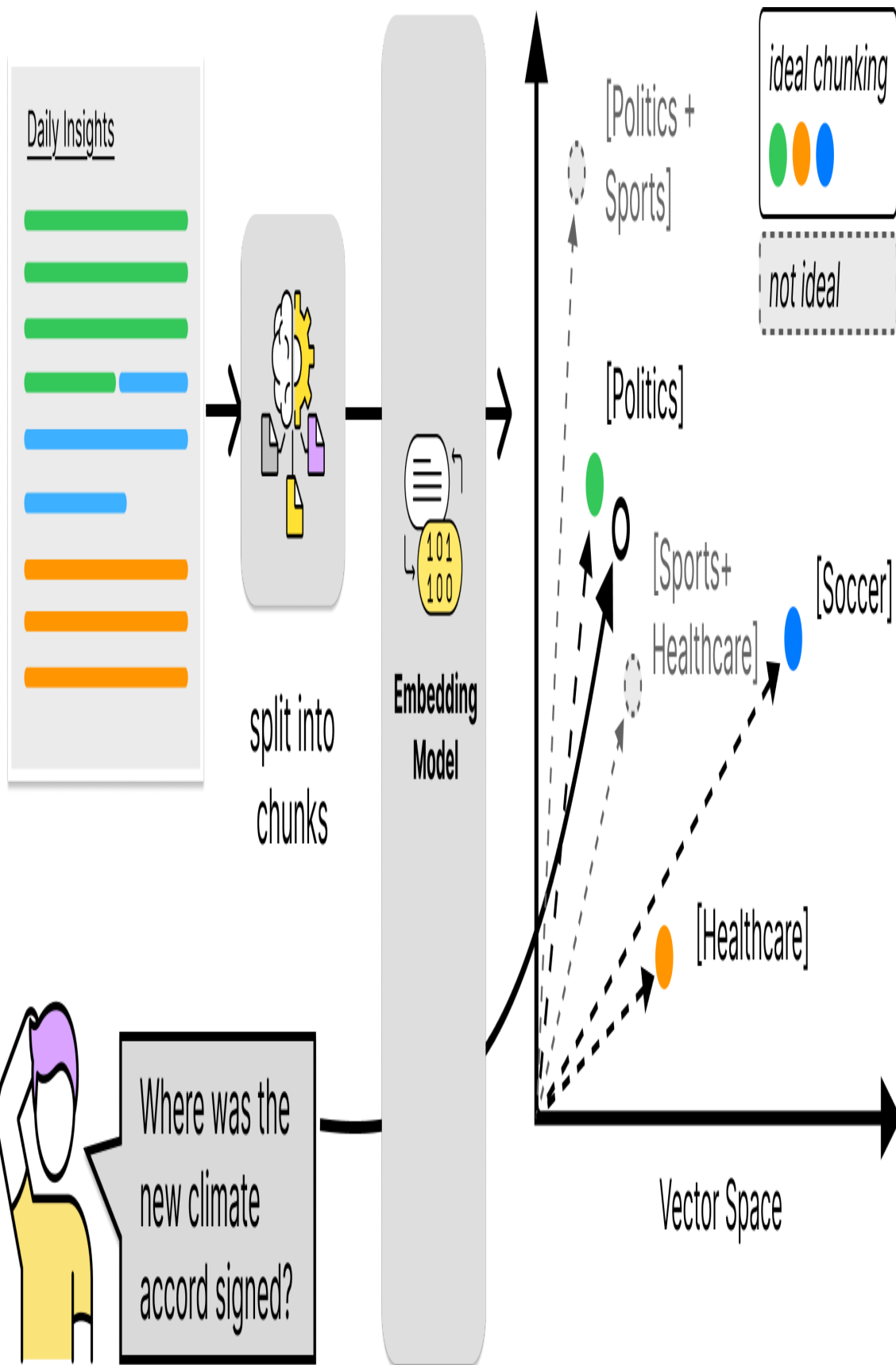


Figure 2-10. Each text chunk should represent one particular piece of information as clearly as possible.

See also

For most documents, the recursive chunker works well. However, for other types of documents, consider these methods:

- For documents created with markup languages like Markdown, HTML, or LaTeX, use document-aware chunking, which understands their syntax (see [Recipe 2.6](#)).
- For unstructured documents like audio transcriptions or chat histories, use semantical chunking, which analyzes the content to determine where to split the text (see [Recipe 2.7](#)).
- For complex documents, agentic chunking can combine related information from across the document to create more meaningful text chunks (see [Recipe 2.8](#)).

2.6 Document Aware Splitting

Problem

You need to load documents with specific syntax and split them into chunks based on their structural elements.

Solution

In markdown files, headings start with a hashtag (#), and bullet list items begin with an asterisk (*). To make text bold, use two asterisks (**).

Document-aware splitters understand this syntax and use it to divide the text. While you could create your own [regex](#) rules, it's simpler to use existing libraries that already handle these syntax rules. LangChain

provides specific splitters for different file types, such as Markdown, Python, HTML, and LaTeX. **Figure 2-11** shows examples of this syntax.



Figure 2-11. Sample code example for HTML, Python, Markdown, and LaTeX

Prerequisites

We are using the text splitters from LangChain.

Steps

Example 2-9 determines the file type based on its extension and selects the appropriate loader. The code snippet handles Markdown, LaTeX, and Python files, using the corresponding LangChain splitters to divide the text.

Example 2-9. Document Aware Splitting

```
file_path = "../../datasets/markdown_files/random_md_code.md"
file_extension = os.path.splitext(file_path)[1]

with open(file_path, "r") as file:
    file_text = file.read()

from langchain_text_splitters import (
    PythonCodeTextSplitter,
    LatexTextSplitter,
    MarkdownHeaderTextSplitter,
)

if file_extension == ".py":
    splitter = PythonCodeTextSplitter(chunk_size=500,
chunk_overlap=50)
elif file_extension == ".tex":
    splitter = LatexTextSplitter(chunk_size=500, chunk_overlap=50)
elif file_extension == ".md":
    splitter = MarkdownHeaderTextSplitter(chunk_size=500,
chunk_overlap=50)

chunks = splitter.split_text(file_text)
```

Discussion

Markup languages use specific syntax to define the appearance of the final document.

To effectively chunk these documents, we need to understand their syntax rules. Fortunately, LangChain provides various splitters tailored for different file types, making this process easier.

See Also

For a brief overview of all these splitters, refer to the [LangChain documentation](#).

If you're working with unstructured documents, such as audio transcriptions or chat logs, consider using more advanced chunking methods that analyze the text content. Examples include semantical chunking (see [Recipe 2.7](#)) and agentic chunking (see [Recipe 2.8](#)).

2.7 Splitting the Text Using Semantic Aware Chunkers

Problem

You want to chunk documents without any structure elements like headings and paragraphs.

Solution

The goal of the splitting process is to create text chunks that each cover a single idea. When writing a blog post or book, you naturally structure your text into paragraphs, making it easier to read. This structure is ideal for recursive chunking. But what if you're working with unstructured documents?

Semantic chunking aims to keep related text together, regardless of the document's structure. It analyzes the content itself to determine which sentences belong together. Using embedding models, we can measure the similarity between consecutive sentences and decide if they should be in the same chunk.

Existing libraries, like the [Semantical Chunker](#), are still experimental. Alternatively, you can create your own semantic chunking pipeline; the process is straightforward.

Figure 2-12 shows the steps involved:

1. **Break text:** Break the text into small pieces, like individual sentences.
2. **Generate embeddings:** Generate embeddings for each small text piece using embedding models (see [Link to Come]).
3. **Measure similarity:** Measure the semantic similarity between consecutive text pieces by calculating the distance between their embeddings (see [Link to Come]).
4. **Merge text chunks:** Compare the similarity of consecutive text pieces. If they are similar, merge them into a larger text chunk. Continue adding sentences to this chunk until the similarity between two consecutive pieces drops below a certain threshold. This threshold can be set using methods like percentile, standard deviation, or gradient.

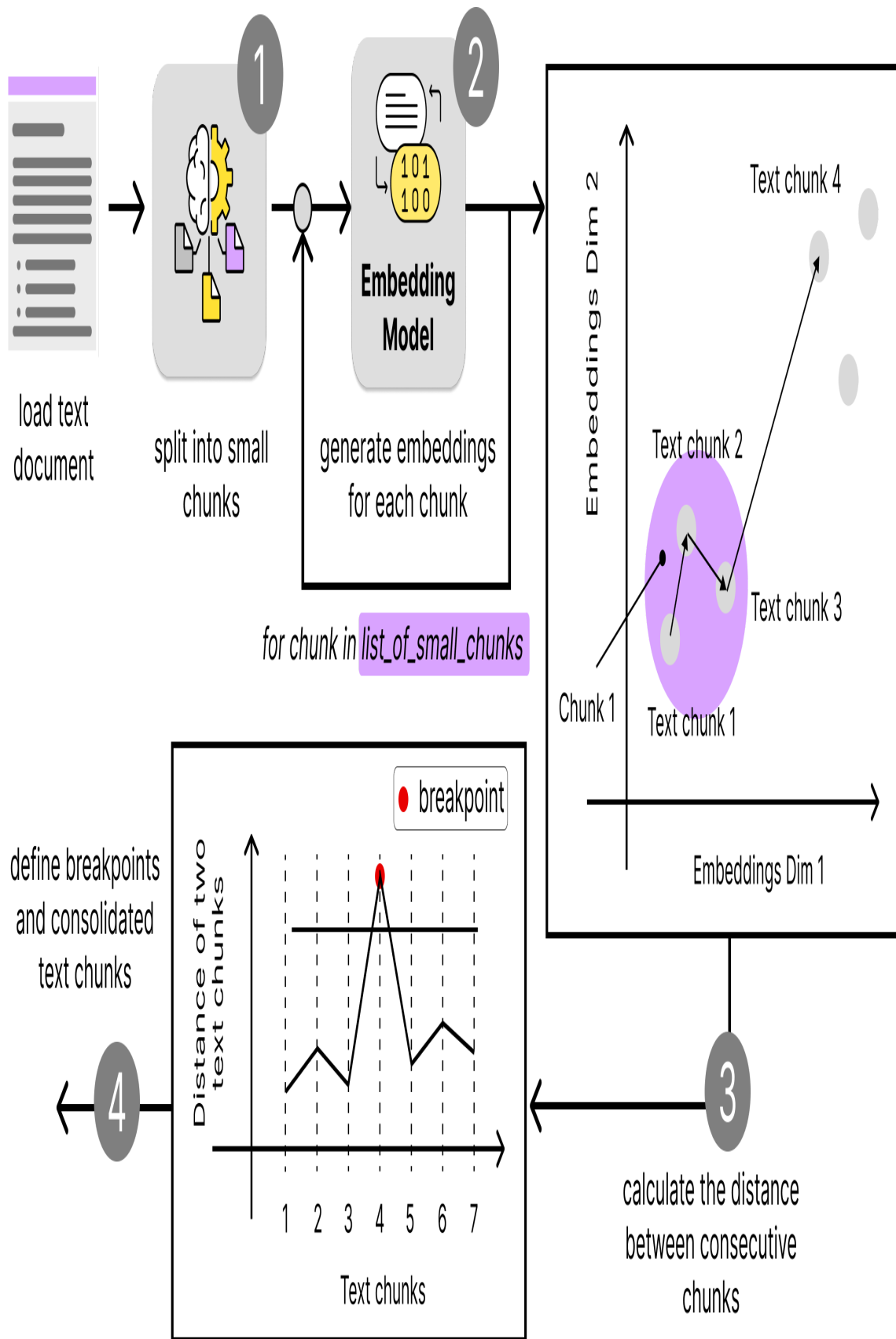


Figure 2-12. Define the breakpoint using the semantical similarity

For the last step, we need to decide what counts as “similar enough.” In this recipe, I’m using the percentile method. We collect all the calculated distances into a list and then determine where to set the breakpoints. Here, we split the text and create a new chunk when the distance exceeds the 90th percentile.

TIP

The distance in the vector space measures how similar two text chunks are. A smaller distance means higher similarity, while a larger distance indicates lower similarity. See [Link to Come] for details on calculating these distances.

Prerequisites

We need an embedding model. In [Example 2-10](#), we will use OpenAI’s Text Embedding model (Ada-003).

To perform the steps mentioned above, we will use LangChain’s [SemanticChunker](#) from the *langchain_experimental* package.

Steps

For the [Example 2-10](#), we use a text sample that lacks structure; it has no headings, paragraphs, or bullet points. The threshold for splitting the text is set to the 90th percentile.

For testing, we will load a [sample news page](#). This is the same text we used in [Recipe 2.5](#) about recursive chunking, but this time all headings and paragraphs have been removed. As humans, we would need to read the text carefully to find meaningful breakpoints. The semantical chunker does this automatically.

Example 2-10. LangChain’s semantical chunker

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai.embeddings import OpenAIEmbeddings
```

```
file_path = "../../datasets/text_files/Latest_News.txt"
```



```
with open(file_path, "r") as file:
    text = file.read()

text_splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type="percentile",
    breakpoint_threshold_amount=90,
)
chunks = text_splitter.split_text(text)
```

When you print the chunks, you will see that it correctly chose the breakpoints at the end of the sports and politics sections. We can adjust the breakpoints in various ways. In [Example 2-10](#), we used the percentile method with a `breakpoint_threshold_type` and set the `breakpoint_threshold_amount` to 90.

The provided text is a simple example. You will need to experiment to find what works best for your documents.

Discussion

Ideally, each text chunk should capture one clear idea, with the next chunk covering a different concept. As mentioned in [Recipe 2.5](#), recursive chunking usually works well for most cases since it uses the document's natural structure, like paragraphs and headings.

However, recursive chunking can be inflexible. It uses separators but does not adjust the chunk size. If we set the chunk size to 500 tokens, it uses the separators to break the chunk but still tries to create chunks with 500 tokens, regardless of whether it makes sense semantically. And what if a proper document structure is missing?

Semantic chunking can also work for texts without any structure. It compares the semantic similarity between two consecutive text chunks. If the similarity is very high, it indicates that the two text chunks belong together.

Although this approach can be costly, it can improve the quality of the text chunks. High-quality text chunks make the retrieval step faster, more cost-

efficient, and more reliable.

While the concept is useful, it may be too complex for simple RAG chatbots. However, having a variety of tools, like the semantical chunker, can be beneficial for different tasks. Machine learning is about creativity, and having an extensive toolkit helps.

See also

For a detailed tutorial on chunking, check out Greg Kamradt's guide on [GitHub](#), which covers 5 levels of chunking, including semantic chunking.

If you prefer an alternative to LangChain, you can explore the [Llama Index semantic chunker](#).

For another approach, consider agentic chunking, which uses LLMs to analyze content. Learn more in [Recipe 2.8](#).

2.8 Splitting Text Using Agentic Chunkers

Problem

You are dealing with long, complex documents and want to bundle related information into meaningful text chunks.

Solution

There isn't a mature library for this yet, but the concept is straightforward. I saw the concept for the first time in Greg Kamradt's Blog Post "[5 Level Of Text Splitting](#)". I highly recommend reading it.

The idea is to create standalone statements called propositions. These propositions are clear and understandable without needing additional context. With the right prompt, the LLM can break down text chunks into clear statements and even add implied ones.

Here's an example:

Original text chunk:

- Sarah bought a new book. She enjoys reading fantasy novels.

Created propositions:

- Sarah bought a new book.
- Sarah enjoys reading fantasy novels.
- Sarah reads fantasy novels in her free time. (implied)

The main idea of agentic chunking is to use these clear statements as text chunks, which can improve the quality and performance of our retrieval step. We can further enhance this by reviewing and merging similar statements.

To do this, we iterate over the propositions. We start with the first proposition to create the initial chunk. For each following proposition, we decide whether to add it to the current chunk or start a new one. This process continues for all propositions. **Figure 2-13** illustrates this step-by-step process.

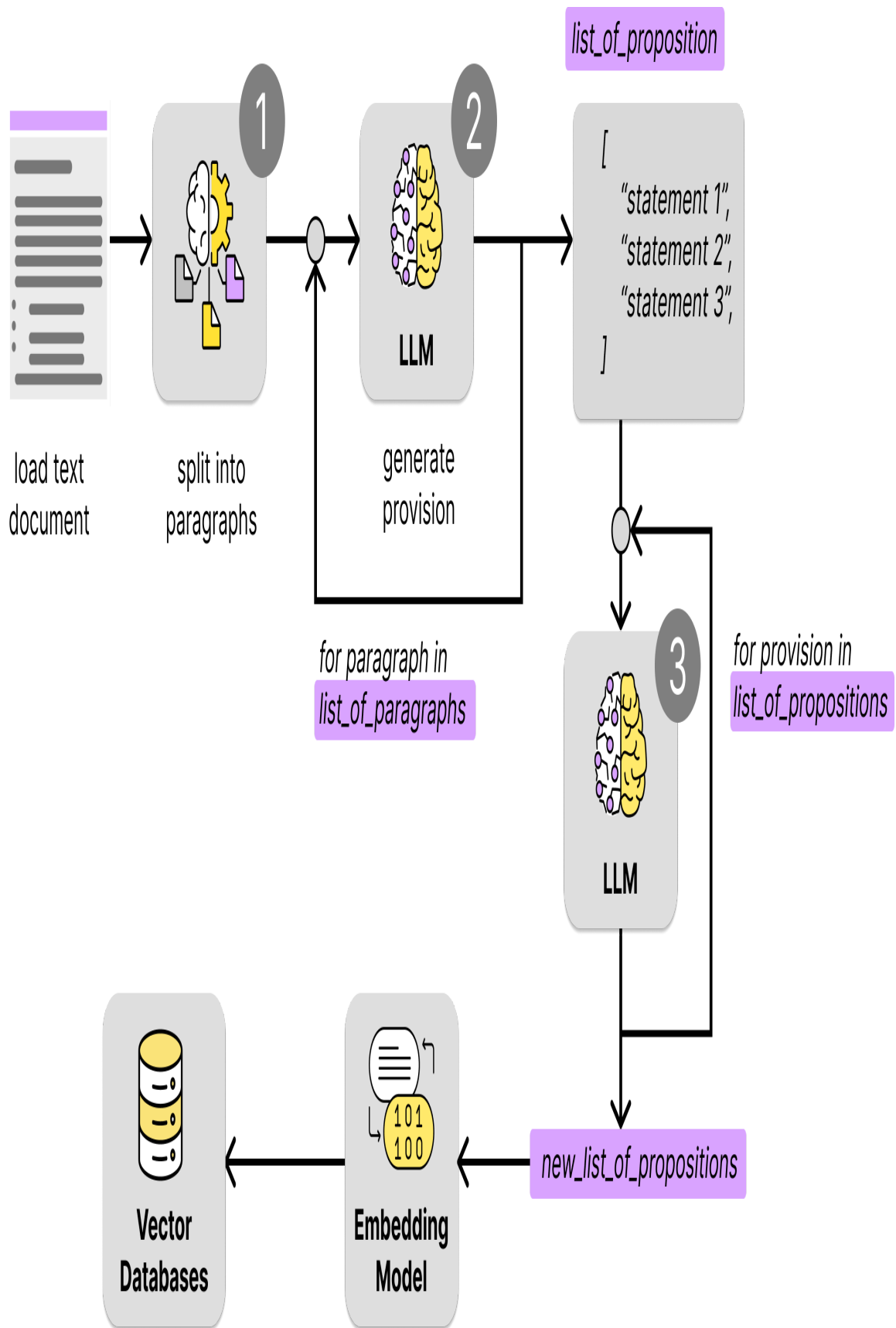


Figure 2-13. How agentic chunkers work

Prerequisites

We need an LLM to generate and compare the propositions. In this example, we are using OpenAI's GPT-4o.

Since we need to process all text chunks multiple times, you might also consider using smaller, more cost-effective models to check if they perform adequately.

Steps

Let's start with the main functions needed to build your own agentic chunking pipeline.

The key step is creating the propositions. In [Example 2-11](#), we generate a runnable with LangChain, which combines the LLM and prompt template. The prompt template we use is from the paper [Dense X Retrieval: What Retrieval Granularity should we use](#). You can find the [prompt](#) in the LangChain hub and load it directly from there. The prompt instructs the LLM to convert the content into "clear and simple" propositions. It provides instructions on preparing the text before saving it in the proposition. The most significant change to the text is that the LLM replaces pronouns, like "he," "she," "they," etc., with the full names of entities.

Example 2-11. How to create propositions

```
from langchain import hub

# pull the prompt template from the langchain hub
obj = hub.pull("wfh/proposal-indexing")

# define the llm you want to use
llm = ChatOpenAI(model="gpt-4o")

# A Pydantic model to extract sentences from the passage
class Sentences(BaseModel):
    sentences: List[str]

extraction_llm = llm.with_structured_output(Sentences)
```

```
# Create the sentence extraction chain
extraction_chain = obj | extraction_llm

# Test it out
sentences = extraction_chain.invoke(
    """
    On July 20, 1969, astronaut Neil Armstrong walked on the moon .
    He was leading the NASA's Apollo 11 mission.
    Armstrong famously said, "That's one small step for man, one
    giant leap for mankind" as he stepped onto the lunar surface.
    """
)
```

The agentic chunker builds a reusable function around the snippet in [Example 2-11](#). You can read the entire document into the model to generate propositions, but this approach lacks control over whether the LLM extracts all relevant statements. To improve performance and maintain control, first chunk the data into relatively large sections.

There is no one-size-fits-all solution; you need to test what works best for you. For example, you can use the recursive splitter from [Recipe 2.5](#) and set the separator to two new lines (`\n\n`). This way, each chunk ends at a paragraph or chapter break.

Send these paragraphs to the LLM to create propositions. You'll get a list of statements for each paragraph. To avoid duplicates and merge similar statements, review all propositions and decide whether to add them to an existing chunk or start a new one.

Discussion

In books, contracts, or any other lengthy documents, information is often scattered throughout, making it tricky to chunk into meaningful segments. Different chapters and clauses are interconnected. As humans, we can piece things together by flipping between chapters or using Google and Wikipedia to clear up any confusion.

In RAG systems, our model only sees content from a single vector search: a collection of text chunks that might help answer a specific question. When

we just chunk the data, we lose the context of how different chapters, paragraphs, and snippets are connected.

Agentic chunkers tackle this issue by using LLMs to create text chunks that merge similar information.

See also

There is no right or wrong. There are multiple approaches out there. You can simply use the standard implementation in LangChain or LlamaIndex, or you come up with your own agentic chunking process, optimized for your use case.

For inspiration, check out the following resources:

- [Greg Kamradt's GitHub repository on chunking strategies](#)
- [Thuwarakesh Murallie's blog post on agentic chunking](#)

[OceanofPDF.com](https://oceanofpdf.com)